

ABHEDYA AI

12-Week Developer Plan

Sovereign AI Workbench for Confidential Industrial Intelligence

Modern Engineering Playbook

Team: Tejas • Dakshit • Nidhish • Samar • Shreya • Safa

CONTENTS

12 weeks • 36 development days

WEEK 01 Project Discovery, Repository Standardization & Local Baseline

- ↳ Friday — Day 1
- ↳ Saturday — Day 2
- ↳ Sunday — Day 3

WEEK 02 Architectural Grounding, API Contracts & State Modeling

- ↳ Friday — Day 4
- ↳ Saturday — Day 5
- ↳ Sunday — Day 6

WEEK 03 Database Foundation, Schema Migrations & Audit Ledger

- ↳ Friday — Day 7
- ↳ Saturday — Day 8
- ↳ Sunday — Day 9

WEEK 04 Local LLM Runtime, Model Registry & Adaptive Router

- ↳ Friday — Day 10
- ↳ Saturday — Day 11
- ↳ Sunday — Day 12

WEEK 05 Multimodal Ingestion Engine & High-Fidelity OCR

- ↳ Friday — Day 13
- ↳ Saturday — Day 14
- ↳ Sunday — Day 15

WEEK 06 Sovereign Vector Retrieval & Hybrid Agentic RAG

- ↳ Friday — Day 16
- ↳ Saturday — Day 17
- ↳ Sunday — Day 18

WEEK 07 LangGraph Agentic Orchestration & Cyclic State Machine

- ↳ Friday — Day 19
- ↳ Saturday — Day 20
- ↳ Sunday — Day 21

WEEK 08 Secure Tool Sandbox, Self-RAG Critique & HITL Checkpoint

- ↳ Friday — Day 22
- ↳ Saturday — Day 23
- ↳ Sunday — Day 24

WEEK 09 Deliverable Synthesis & Frontend Console Real-Time Integration

- ↳ Friday — Day 25
- ↳ Saturday — Day 26
- ↳ Sunday — Day 27

WEEK 10 Zero-Egress Network Sentinel & Master Containerization

- ↳ Friday — Day 28
- ↳ Saturday — Day 29
- ↳ Sunday — Day 30

WEEK 11 Security Hardening, Resilience Testing & SIH Rehearsal

- ↳ Friday — Day 31
- ↳ Saturday — Day 32
- ↳ Sunday — Day 33

WEEK 12 Individual Verification, Technical Defense & Handover

- ↳ Friday — Day 34
- ↳ Saturday — Day 35
- ↳ Sunday — Day 36

Final Project Completion Checklist

WEEK 01

Project Discovery, Repository Standardization & Local Baseline

FRIDAY

Day 1

Friday — Day 1

Goal

Establish universal repository access, enforce Git branch protection rules, configure pre-commit hooks, and audit every team member's local workstation against the baseline development requirements.

Task Sequence

1. DEV-001.1 [Shreya & Safa]: Audit workstation hardware and operating systems across all 6 developers (RAM $\geq$ 16GB, disk headroom $\geq$ 40GB, CPU/GPU capabilities).
2. DEV-001.2 [Safa & Dakshit]: Verify environment prerequisites on all 6 laptops (Git, Node.js, Python, curl, OpenSSL); establish the team's shared Friday sync notes template.
3. DEV-001.3 [Tejas & Samar]: Configure GitHub repository access, SSH authentication keys, and create individual working branches (tejas, dakshit, nidhish, samar, shreya, safa) off main.
4. DEV-001.4 [Samar]: Configure repository branch protection rules on main (require PR review, block force pushes, require passing CI status checks); define Conventional Commit standards.
5. DEV-001.5 [Tejas]: Configure Git pre-commit hooks for formatting (prettier, eslint, black, flake8) and secret detection (detect-secrets).
6. DEV-001.6 [Nidhish]: Verify Python runtime configurations (Python 3.11); audit .gitignore to guarantee local LLM model weights (.bin, .gguf, ollama/) are strictly excluded.
7. DEV-001.7 [Dakshit]: Clone repository via SSH; test pre-commit formatting on sample TypeScript files; document common Git troubleshooting commands in developer_plan.md.

Developer Responsibilities (In Execution Order)

1. Shreya (Hardware Audit & Security Scanning): Initiates the day by auditing all 6 laptops for hardware minimums and running historical commit scans to ensure zero credentials exist in Git history.
2. Safa (Environment Prerequisite Verification): Follows up immediately by verifying Git, Python, and Node installations on every machine and drafting the team's Friday sync notes template.
3. Tejas (Repository Setup & Client Pre-commit): Configures repository settings, creates developer working branches, verifies SSH keys for Dakshit and Safa, and sets up Prettier/ESLint pre-commit hooks.
4. Samar (Branch Protection & Commit Governance): Enforces branch protection on main, defines Conventional Commit standards (feat:, fix:, docs:, test:), and tests policy by verifying direct pushes are blocked.
5. Nidhish (Runtime Validation & Model Weight Exclusion): Audits Python 3.11 configurations across machines and updates .gitignore to prevent multi-gigabyte Ollama model weights from being tracked.
6. Dakshit (SSH Clone Verification & Documentation): Clones the repository via SSH, tests the pre-commit hooks on sample code, and publishes the Git troubleshooting cheat-sheet in developer_plan.md.

Collaboration Flow

Shreya and Safa kick off the day by auditing team hardware. Tejas and Samar then pair on repository branch policies and commit standards. Nidhish and Dakshit verify that pre-commit hooks and .gitignore behave identically across Windows PowerShell and Linux bash environments.

Prerequisites

- GitHub accounts registered for all 6 members.
- Git installed locally (`git --version` $\geq$ 2.40).
- Terminal access with SSH client enabled.

Hands-On Requirement

All 6 developers must generate an SSH keypair, add the public key to GitHub, clone the repository, checkout their named branch, make a test commit adhering to Conventional Commits, and push successfully to `origin/<name>`.

Knowledge Sharing

- Presenter: Tejas & Samar
- Topic: "Git Branching Strategy, Pre-Commit Hygiene & Preventing Accidental Secret Commits in Industrial Codebases."
- Duration: 25 minutes during the offline Friday sync.

Review

Shreya reviews commit logs to ensure no credentials were pushed; Samar verifies branch protection rules block direct commits to `main`.

Documentation

Update `developer_plan.md` with the verified Git workflow, branch naming conventions, and team SSH setup guide.

Expected Result

Repository cloned on all 6 developer machines; branch protection active on `main`; pre-commit hooks rejecting malformed commits.

Definition of Done

All 6 developers have an active working branch, have pushed a verified signed commit, and have passed pre-commit hook execution.

SATURDAY

Day 2

Saturday — Day 2

Goal

Install and configure the complete local runtime ecosystem (Python 3.11 virtual environment, Node.js 20 LTS, Ollama SLM engine) on every developer's machine and eliminate environment drift.

Task Sequence

1. DEV-002.1 [Samar & Dakshit]: Standardize Python 3.11 installation across all 6 systems; create isolated virtual environments (`backend/venv`).
2. DEV-002.2 [Samar]: Install backend dependencies (`fastapi`, `uvicorn`, `sqlalchemy`, `pydantic`, `python-docx`, `psutil`, `pytest`) from `backend/requirements.txt`.
3. DEV-002.3 [Tejas]: Standardize Node.js 20 LTS and install frontend packages (`npm install` inside `frontend/`) using Next.js 16 and TailwindCSS.
4. DEV-002.4 [Nidhish]: Install and start local Ollama engine (`http://localhost:11434`); pull baseline lightweight test model (`qwen2.5:1.5b` or `qwen2.5:7b`).
5. DEV-002.5 [Shreya]: Audit `.env` files across all systems to guarantee environment isolation; confirm that `DATABASE_URL` and `STORAGE_PATH` point to local sovereign directories.
6. DEV-002.6 [Safa]: Execute step-by-step installation on her laptop as the baseline test; log every terminal warning and publish the official troubleshooting guide.

Developer Responsibilities (In Execution Order)

1. Samar (Backend Environment Lead): Starts by establishing the clean Python 3.11 virtual environment and installing backend packages, verifying zero version conflicts between SQLAlchemy and Pydantic v2.
2. Dakshit (Virtualenv Co-lead & Cross-Platform Tester): Assists Safa and Shreya with Python virtual environment creation; verifies `pip install` commands on Windows PowerShell and Linux.
3. Tejas (Frontend Tooling Lead): Guides Node.js 20 LTS standardization and executes `npm install` across all machines, verifying that Next.js 16 compiles without warnings.
4. Nidhish (Local AI Engine Lead): Leads the Ollama installation session; tests loopback connectivity (`curl http://localhost:11434/api/tags`); verifies model pulling script `scripts/pull_models.sh`.
5. Shreya (Environment Security Auditor): Audits `.env` configuration files across all 6 systems, ensuring paths point strictly to local disks and no external endpoints are referenced.
6. Safa (End-to-End Environment Validator): Replicates the entire installation sequence on her laptop, documenting edge cases, OS-specific quirks, and authoring the setup troubleshooting guide.

Collaboration Flow

Samar sets up the Python foundation; Tejas brings up the Node.js frontend packages; Nidhish screen-shares Ollama setup and model pulling; Shreya audits configuration security; Dakshit and Safa validate the setup across all machines.

Prerequisites

- DEV-001 completed.
- Python 3.11 installed.
- Node.js 20.x installed.
- Administrative privileges to install Ollama.

Hands-On Requirement

Every developer must personally run:

```
# Backend
cd backend && python -m venv venv && source venv/bin/activate # or .\venv\Scripts\Activate.ps1
pip install -r requirements.txt
# Frontend
cd ../frontend && npm install
# Ollama
ollama pull qwen2.5:1.5b && curl http://localhost:11434/api/tags
```

Knowledge Sharing

- Presenter: Nidhish
- Topic: "Local AI Inference with Ollama: Loopback Networking, Memory Allocation & Managing VRAM Headroom."
- Duration: 30 minutes during the Saturday online session.

Review

Samar checks that all 6 backend virtual environments activate cleanly; Tejas inspects that `npm run build` succeeds without package errors.

Documentation

Document the complete environment setup steps, required software versions, and OS-specific fixes in `docs/10-development/local-setup.md` and `developer_plan.md`.

Expected Result

Identical Python 3.11, Node.js 20, and Ollama runtimes functioning on all 6 machines with zero external cloud dependencies.

Definition of Done

All 6 developers execute `python -c "import fastapi, sqlalchemy, pydantic; print('Backend OK')"` and `node -v` successfully, with Ollama returning local model tags.

Sunday — Day 3

Goal

Execute the first end-to-end baseline run of both the FastAPI backend and Next.js frontend across all 6 laptops, perform an architectural reality audit, and log existing codebase gaps.

Task Sequence

1. DEV-003.1 [Samar]: Boot FastAPI backend (`uvicorn app.main:app --reload --port 8000`) and verify `/health` endpoint returns 200 OK.
2. DEV-003.2 [Tejas]: Boot Next.js frontend (`npm run dev -- -p 3000`) and verify browser opens the Sovereign AI Workbench console at `http://localhost:3000`.
3. DEV-003.3 [Nidhish]: Test backend LLM fallback execution in `app/api/chat.py`; verify how prompt classification interacts with Ollama.
4. DEV-003.4 [Dakshit]: Inspect `frontend/src/lib/api.ts` and `frontend/src/store/useTaskStore.ts`; document how task states and streaming responses are received in the UI.
5. DEV-003.5 [Shreya]: Run `backend/app/network_sentinel/monitor.py`; inspect socket telemetry on `GET /api/network/status`; verify that no outbound packets leave the system during prompt execution.
6. DEV-003.6 [Safa]: Maintain the live audit matrix; record which features work, which are prototypes, and which are missing; compile findings into Week 1 Summary.

Developer Responsibilities (In Execution Order)

1. Samar (Backend Server Dispatcher): Starts the FastAPI application gateway on port 8000, demonstrates the OpenAPI `/docs` interface, and walks through the SSE event broadcaster and database models.
2. Tejas (Frontend Console Dispatcher): Boots the Next.js frontend on port 3000, demonstrates the 3-column AppShell, and triggers the first live REST call to the backend.
3. Nidhish (LLM Loopback Tester): Submits test engineering prompts to `POST /api/chat`, observes Ollama prompt processing in terminal logs, and verifies response generation.
4. Dakshit (State & Stream Inspector): Traces how backend response streams update the Zustand store in the frontend, identifying mock components that need live backend feeds.
5. Shreya (Zero-Egress Socket Verifier): Executes socket monitoring during chat generation, verifying that all network packets stay bound to `127.0.0.1` and zero external connections are opened.
6. Safa (Audit Log Recorder & Scribe): Compiles the gap matrix comparing current code against the authoritative 9-layer architecture, publishing the official Week 1 audit report.

Collaboration Flow

Samar starts the backend; Tejas starts the frontend; Nidhish sends the first test prompt; Dakshit traces the UI state; Shreya verifies network confinement; Safa records gaps and consolidates the sprint log.

Prerequisites

- DEV-001 and DEV-002 completed.
- Ports 8000, 3000, and 11434 available.

Hands-On Requirement

All 6 developers must independently open `http://localhost:3000`, see the industrial workbench UI, send a test chat message, see backend terminal logs processing the request, and receive an SSE streamed response.

Knowledge Sharing

- Presenter: Samar & Tejas
- Topic: "Sovereign Architecture Walkthrough: How the Next.js UI, FastAPI Gateway, and Local Ollama Runtime Communicate."
- Duration: 35 minutes during the Sunday online session.

Review

Cross-verification: Tejas verifies Dakshit's and Safa's UI execution; Samar verifies Shreya's and Nidhish's backend execution.

Documentation

Commit the Week 1 Reality Audit Table into docs/changeLog.md and update developer_plan.md with Week 1 sign-offs.

Expected Result

Every developer has a fully operational local baseline running on their machine, understands the architectural topology, and knows exactly what gaps need to be built.

Definition of Done

100% of the team (all 6 developers) demonstrates a working local instance of frontend and backend communicating via REST and SSE.

WEEK 02

Architectural Grounding, API Contracts & State Modeling

FRIDAY

Day 4

Friday — Day 4

Goal

Deep-dive into the 9-layer authoritative ABHEDYA AI architecture, establish shared terminology, and formalize the core WorkbenchState Pydantic model that flows through the system.

Task Sequence

1. DEV-004.1 [Samar]: Conduct architecture review session covering all 9 layers (UI $\rightarrow$ Gateway $\rightarrow$ Sentinel $\rightarrow$ Router $\rightarrow$ Policy $\rightarrow$ LangGraph $\rightarrow$ Tools/RAG $\rightarrow$ Critique $\rightarrow$ Human Sign-off).
2. DEV-004.2 [Dakshit]: Map multimodal state fields (ocr_tables, vlm_findings, extracted_measurements) ensuring raw and structured document data can be serialized.
3. DEV-004.3 [Nidhish]: Define routing and LLM state fields (selected_model, routing_rationale, llm_raw_response, reasoning_tokens); map prompt schema inputs.
4. DEV-004.4 [Safa]: Map tool execution and critique fields (tool_plan, sandbox_result, isrel_score, issup_score, critique_status); document field definitions.
5. DEV-004.5 [Tejas]: Map frontend rendering requirements into state fields (node step states, router confidence cards, citation card excerpts).
6. DEV-004.6 [Samar]: Lead the construction of backend/app/schemas/state.py; implement Pydantic v2 BaseModel with validation rules; verify JSON serialization.
7. DEV-004.7 [Shreya]: Review state schema for security and confidentiality; ensure sensitive operator credentials and file paths are sanitized; verify audit-trail tracking fields (state_hash, parent_step_id).

Developer Responsibilities (In Execution Order)

1. Samar (System Architecture Lead): Walks the team through the 9-layer data lifecycle of an NDT report and scaffolds backend/app/schemas/state.py.
2. Dakshit (Multimodal State Architect): Supplies field definitions for OCR table matrices and VLM visual defect observations.
3. Nidhish (LLM & Router State Architect): Supplies state fields for model selection, routing justification, and reasoning tokens.
4. Safa (Critique & Sandbox State Architect): Supplies state fields for code execution results, ISREL/ISSUP critique scores, and revision loop counters.
5. Tejas (Frontend State Alignment): Reviews state fields against UI display components, ensuring all telemetry items can be extracted cleanly.
6. Shreya (Security & Immutability Auditor): Enforces data immutability, audits sanitization of local file paths, and validates cryptographic tracking fields.

Collaboration Flow

Samar leads the architecture review and screen-shares the code. Dakshit, Nidhish, and Safa provide specialized field requirements. Tejas ensures client compatibility. Shreya signs off on security and immutability.

Prerequisites

- Week 1 completed.
- Understanding of ADR-001 to ADR-008.

Hands-On Requirement

Every developer pulls the newly created feature/state-models branch, inspects backend/app/schemas/state.py, and runs the Pydantic validation unit test (pytest backend/tests/test_state_schema.py).

Knowledge Sharing

- Presenter: Samar & Shreya
- Topic: "State Machine Theory & Pydantic Data Modeling: Why Immutability and Strict Typing Prevent Agent Drift."
- Duration: 25 minutes during Friday offline sync.

Review

Nidhish reviews state fields for LLM compatibility; Tejas reviews state fields for TypeScript interface alignment.

Documentation

Document the WorkbenchState data dictionary and field lifecycle diagram in docs/05-system-design/orchestrator-langgraph.md and developer_plan.md.

Expected Result

A single, validated WorkbenchState Pydantic class merged to the shared development branch, ready for LangGraph and API consumption.

Definition of Done

backend/app/schemas/state.py compiles with zero Pydantic warnings and passes 100% of schema validation tests.

SATURDAY

Day 5

Saturday — Day 5

Goal

Define and implement strict REST API contracts and Server-Sent Event (SSE) specifications, matching backend Pydantic models with frontend TypeScript interfaces.

Task Sequence

1. DEV-005.1 [Samar]: Finalize REST API endpoints: POST /api/chat, POST /api/upload, GET /api/tasks/{id}, POST /api/tasks/{id}/approve, GET /api/tasks/{id}/download, GET /api/network/status.
2. DEV-005.2 [Samar]: Implement standardized Pydantic request and response schemas in backend/app/schemas/api.py.
3. DEV-005.3 [Tejas]: Create mirroring TypeScript interfaces in frontend/src/types/api.ts and frontend/src/types/task.ts; update frontend/src/lib/api.ts.
4. DEV-005.4 [Dakshit]: Align upload contracts (POST /api/upload); ensure multipart form data handling supports .pdf, .docx, .xlsx, and image MIME types with size boundaries.
5. DEV-005.5 [Nidhish]: Define /api/chat payload parameters (temperature, model preference, system prompt overrides); ensure fallback responses follow structured error schemas.
6. DEV-005.6 [Shreya]: Implement network status response contracts (/api/network/status); define security response headers (Content-Security-Policy, anti-sniffing, zero-cache).
7. DEV-005.7 [Safa]: Write comprehensive API contract test suite (backend/tests/test_api_contracts.py) validating response status codes, payload structures, and error states.

Developer Responsibilities (In Execution Order)

1. Samar (API Gateway Architect): Authors the core route definitions and Pydantic schemas in `backend/app/api/` and exports the generated OpenAPI specification (`openapi.json`).
2. Tejas (TypeScript Client Architect): Consumes the OpenAPI schema, builds typed API methods in `frontend/src/lib/api.ts`, and establishes typed SSE event listeners.
3. Dakshit (Upload Contract Specialist): Validates multipart file upload endpoints, boundary limits (25MB), and file metadata response schemas.
4. Nidhish (Chat Payload Specialist): Validates request parameters for local model inference and structures standardized error payloads.
5. Shreya (Security Header & Telemetry Specialist): Enforces strict HTTP security headers and standardizes the `/api/network/status` response schema.
6. Safa (API Test Harness Lead): Authors and runs automated contract tests (`pytest backend/tests/test_api_contracts.py`), verifying every status code and payload structure.

Collaboration Flow

Samar establishes the server endpoints; Tejas immediately mirrors them into client TypeScript interfaces; Dakshit and Nidhish refine upload and chat parameters; Shreya secures headers; Safa executes the contract test suite.

Prerequisites

- DEV-004 completed.
- Understanding of OpenAPI 3.0 standards.

Hands-On Requirement

All 6 developers must execute `pytest backend/tests/test_api_contracts.py` and run `npm run type-check` in frontend to verify that zero type mismatches exist between client and server.

Knowledge Sharing

- Presenter: Tejas & Dakshit
- Topic: "Contract-First API Design: Eliminating Client-Server Mismatches with OpenAPI and TypeScript Type Generation."
- Duration: 30 minutes during Saturday online session.

Review

Dakshit reviews backend schemas; Nidhish reviews frontend TypeScript interfaces; Samar reviews error handling logic.

Documentation

Publish complete API specification in `docs/09-apis/api-reference.md` and `docs/09-apis/sse-event-specs.md`.

Expected Result

Locked, battle-tested API contracts and SSE event specifications with automated type-checking on both frontend and backend.

Definition of Done

All REST routes return validated schemas; frontend compiles cleanly against TypeScript API types; `test_api_contracts.py` passes 100%.

SUNDAY

Day 6

Sunday — Day 6

Goal

Implement a simulated end-to-end integration test harness where the frontend sends requests to the backend, receives real-time SSE stream events with mock agent traces, and demonstrates the Human-in-the-Loop pause/resume flow.

Task Sequence

1. DEV-006.1 [Samar]: Refactor backend/app/core/sse_manager.py to support multi-client subscription queues and connection heartbeat pings.
2. DEV-006.2 [Samar & Nidhish]: Create an integration test harness (scripts/simulate_agent_flow.py) emitting realistic sequential agent steps (Ingest $\rightarrow$ Route $\rightarrow$ OCR $\rightarrow$ RAG $\rightarrow$ Sandbox $\rightarrow$ Critique $\rightarrow$ HITL).
3. DEV-006.3 [Tejas]: Wire frontend ContextPanel.tsx and ExecutionTimeline.tsx to display real-time node state transitions (active, completed, error).
4. DEV-006.4 [Dakshit]: Test frontend state retention during streaming; ensure the UI handles network reconnections and does not duplicate message bubbles.
5. DEV-006.5 [Shreya]: Monitor socket behavior during continuous SSE streaming; verify no memory leaks or socket leaks occur using psutil.
6. DEV-006.6 [Safa]: Execute the end-to-end test flow on her machine; capture screenshots of the active timeline and approval modal; update documentation.

Developer Responsibilities (In Execution Order)

1. Samar (SSE Event Broadcaster): Implements the pub-sub connection queue in sse_manager.py with 15-second heartbeat pings.
2. Nidhish (Trace Payload Synthesizer): Generates realistic simulated agent step payloads (thickness numbers, API clauses, severity ratings) for the simulation script.
3. Tejas (UI Stream Visualizer): Connects the frontend ExecutionTimeline to the live SSE stream, animating node transitions as events arrive.
4. Dakshit (Stream Resilience Tester): Simulates connection drops and rapid UI refreshes to ensure no duplicate message bubbles appear in the chat feed.
5. Shreya (Socket & Leak Auditor): Runs memory and socket audits during the stream, verifying that disconnected SSE clients are cleanly purged from server memory.
6. Safa (End-to-End Verifier & Documenter): Runs the complete simulation across all machines, logs timings, takes UI screenshots, and updates the testing documentation.

Collaboration Flow

Samar and Nidhish launch the backend trace generator; Tejas connects the frontend UI; Dakshit tests client resilience; Shreya audits server sockets; Safa performs cross-laptop verification.

Prerequisites

- DEV-004 and DEV-005 completed.
- Backend and frontend running locally.

Hands-On Requirement

All 6 developers must execute the simulation script, view the animated node timeline in their local browser, click "Approve" in the approval modal, and verify the task marks as "Completed".

Knowledge Sharing

- Presenter: Samar & Tejas
- Topic: "Server-Sent Events (SSE) vs WebSockets: Why Unidirectional Streaming is Superior for Air-Gapped Agent Telemetry."
- Duration: 25 minutes during Sunday online session.

Review

Nidhish and Dakshit review frontend state handling; Shreya and Safa review backend SSE connection cleanup.

Documentation

Record the verified integration workflow in docs/11-testing/testing-strategy.md and update developer_plan.md Week 2 milestone.

Expected Result

A fully functional simulated communication loop between frontend and backend proving that the contract, SSE stream, and approval pause work seamlessly.

Definition of Done

All 6 developers independently verify the full streaming cycle with human approval on their own machines without errors.

WEEK 03

Database Foundation, Schema Migrations & Audit Ledger

FRIDAY

Day 7

Friday — Day 7

Goal

Design, deploy, and migrate the production PostgreSQL 16 relational database schema, replacing SQLite fallback with enterprise-grade relational storage and Alembic migrations.

Task Sequence

- DEV-007.1 [Samar]: Install PostgreSQL 16 locally (or launch official PostgreSQL container on port 5432); create database `abhedya_db` and role `abhedya_user`.
- DEV-007.2 [Samar & Dakshit]: Design complete relational schema DDL: `tasks`, `agent_steps`, `documents`, `approvals`, `deliverables`, and `audit_logs`.
- DEV-007.3 [Samar]: Initialize Alembic migration environment (`backend/alembic/`) and configure `env.py` to bind with SQLAlchemy ORM models.
- DEV-007.4 [Shreya]: Review database security: enforce least-privilege connection strings, non-root user execution, and password hashing in `.env`.
- DEV-007.5 [Tejas & Nidhish]: Review schema relationships to ensure task history, document listings, and step outputs can be queried efficiently with pagination.
- DEV-007.6 [Safa]: Test Alembic migrations on Windows and Linux; test clean rollback (`alembic downgrade -1`) and re-upgrade; document setup guide.

Developer Responsibilities (In Execution Order)

- Samar (Database Architect): Configures the PostgreSQL instance, writes the SQLAlchemy ORM models in `backend/app/db/models.py`, and initializes Alembic.
- Dakshit (Data Model Contributor): Defines table schemas for document storage, foreign keys, and file metadata relationships.
- Shreya (Database Security Lead): Enforces secure credentials, sets local-only bind rules, and verifies connection string encryption.
- Nidhish (JSONB Schema Reviewer): Validates that `agent_steps.input_data` and `output_data` JSONB columns accept arbitrary model reasoning outputs without schema alterations.
- Tejas (Query Performance Reviewer): Ensures appropriate database indexes exist on `task_id` and `created_at` to support fast UI pagination.
- Safa (Migration Execution & Rollback Tester): Applies migrations across different developer OS platforms, tests downgrade scripts, and publishes migration guides.

Collaboration Flow

Samar and Dakshit build the SQL foundation; Shreya secures connection profiles; Nidhish and Tejas verify querying needs; Safa executes and tests migration rollbacks.

Prerequisites

- Week 2 completed.

- PostgreSQL 16 installed or Docker desktop running.

Hands-On Requirement

Every developer must personally run:

```
# Initialize DB and apply migrations
createdb -U postgres abhedya_db
cd backend && alembic upgrade head
# Verify tables exist
psql -U postgres -d abhedya_db -c "\dt"
```

Knowledge Sharing

- Presenter: Samar & Shreya
- Topic: "PostgreSQL Relational Design & Alembic Migrations: Ensuring Zero Data Loss Across Enterprise Refactoring."
- Duration: 25 minutes during Friday offline sync.

Review

Dakshit verifies table structures; Nidhish tests JSONB column queries; Samar validates foreign key cascade constraints.

Documentation

Document database schema diagrams, table definitions, and migration commands in docs/06-data/database-schema.md and developer_plan.md.

Expected Result

PostgreSQL 16 active on all 6 developer machines with all 6 core relational tables created via Alembic migrations.

Definition of Done

alembic upgrade head runs cleanly on all 6 machines, creating all tables with foreign keys and indexes verified.

SATURDAY
Y

Day 8

Saturday — Day 8

Goal

Implement clean, test-driven database CRUD services and repository patterns for tasks, steps, documents, and approvals.

Task Sequence

1. DEV-008.1 [Samar]: Implement database session context manager (get_db) ensuring automatic commit on success and rollback on exception.
2. DEV-008.2 [Samar]: Build TaskRepository methods (create_task, update_task_status, get_by_id, list_tasks) with strict status transition checks.
3. DEV-008.3 [Dakshit]: Implement DocumentRepository methods (create_document, get_by_task_id, list_documents); test file metadata persistence.
4. DEV-008.4 [Nidhish]: Implement StepRepository methods (create_step, get_steps_for_task); verify JSONB serialization for complex LLM outputs.
5. DEV-008.5 [Safa]: Author test cases for approval records (create_approval); verify operator signatures and decision notes store cleanly.
6. DEV-008.6 [Shreya]: Test transaction rollbacks: simulate backend crash during multi-table insert and verify database remains in consistent state.
7. DEV-008.7 [Tejas]: Test repository methods against frontend queries; verify that GET /api/tasks returns paginated task history with correct timestamps.

Developer Responsibilities (In Execution Order)

1. Samar (Repository Pattern Lead): Builds the core transaction manager and TaskRepository, enforcing legal status transitions (pending $\rightarrow$ processing $\rightarrow$ completed).
2. Dakshit (Document Persistence Specialist): Builds DocumentRepository for linking uploaded and synthesized files to active tasks.
3. Nidhish (Agent Step Persistence Specialist): Builds StepRepository handling JSONB serialization of model thoughts, tool inputs, and outputs.
4. Safa (Approval Persistence Specialist): Builds repository methods for recording human sign-off actions, operator IDs, and review notes.
5. Shreya (Transactional Integrity Auditor): Authors crash and rollback test cases, validating that failed transactions leave zero orphaned records.
6. Tejas (Client Query Integrator): Connects repository outputs to API route handlers, verifying paginated task history displays properly in UI.

Collaboration Flow

Samar, Dakshit, and Nidhish write repository classes; Safa writes the approval persistence; Shreya executes rollback stress testing; Tejas verifies API responses.

Prerequisites

- DEV-007 completed.
- PostgreSQL running locally.

Hands-On Requirement

All 6 developers execute `pytest backend/tests/test_database_crud.py` on their local machines, verifying 100% test pass rate across all repository methods.

Knowledge Sharing

- Presenter: Dakshit & Samar
- Topic: "The Repository Pattern in FastAPI: Decoupling SQL Operations from Business Logic for High Reliability."
- Duration: 30 minutes during Saturday online session.

Review

Samar reviews repository code for SQL injection vulnerabilities; Shreya reviews session scoping to prevent connection pool exhaustion.

Documentation

Update `docs/06-data/database-schema.md` with repository method signatures and test instructions in `developer_plan.md`.

Expected Result

Modular, fully tested repository layer decoupling the FastAPI routes from direct SQL queries, running against PostgreSQL.

Definition of Done

All CRUD repository methods pass 100% of automated unit tests with clean transactional rollbacks verified.

SUNDAY

Day 9

Sunday — Day 9

Goal

Implement the tamper-evident, forward-chained SHA-256 Immutable Audit Ledger in PostgreSQL, preventing silent log manipulation and guaranteeing legal non-repudiation.

Task Sequence

1. DEV-009.1 [Shreya]: Implement cryptographic forward-chaining algorithm ($H_n = \text{SHA-256}(H_{n-1} + \text{Payload}_n)$) in `backend/app/core/audit_ledger.py`.
2. DEV-009.2 [Samar]: Create database triggers/listeners appending audit events on every state transition, file upload, and human approval.
3. DEV-009.3 [Dakshit]: Test audit event payload generation for document uploads; ensure document SHA-256 hashes are bound into the audit chain.
4. DEV-009.4 [Nidhish]: Test audit event generation for model routing decisions; ensure model name, prompt tokens, and rationale are immutably logged.
5. DEV-009.5 [Shreya]: Build audit verification utility (`backend/app/core/audit_verifier.py`) that recalculates the hash chain from block 0 to verify ledger integrity.
6. DEV-009.6 [Safa]: Build deliberate tampering test script: modify a historical database row and prove the verifier immediately detects and pinpoints the corrupted row.
7. DEV-009.7 [Tejas]: Build frontend `AuditTrailView.tsx` displaying the immutable hash chain, sequence numbers, event types, and a visual "Chain Verified" badge.

Developer Responsibilities (In Execution Order)

1. Shreya (Cryptographic Architect): Formulates the SHA-256 forward-chaining mathematical algorithm and implements the ledger generation and verification engine.
2. Samar (Database Event Hook Engineer): Hooks the audit logger into the database repository layer, ensuring every task transition automatically appends a block.
3. Dakshit (Document Audit Specialist): Formats document upload audit payloads, hashing binary files and binding them to the ledger.
4. Nidhish (AI Decision Audit Specialist): Formats model routing and prompt audit payloads, ensuring AI reasoning is recorded for compliance.
5. Safa (Tampering Test Engineer): Writes `scripts/test_audit_tampering.py`, simulates an attacker altering a past inspection record, and verifies detection.
6. Tejas (Audit Interface Engineer): Builds the `AuditTrailView` in the frontend, rendering the sequence of blocks with real-time verification indicators.

Collaboration Flow

Shreya establishes the cryptographic core; Samar wires it to database transactions; Dakshit and Nidhish supply event payloads; Safa tests tampering detection; Tejas visualizes the ledger in the UI.

Prerequisites

- DEV-007 and DEV-008 completed.
- Python `hashlib` available.

Hands-On Requirement

Every developer must run `python scripts/test_audit_tampering.py`, observe the script modify a database row, and verify that `audit_verifier` correctly flags the tampering and identifies the exact sequence number.

Knowledge Sharing

- Presenter: Shreya
- Topic: "Forward-Chained Cryptographic Audit Ledgers: Proving System Integrity to Statutory Inspectors Without Blockchain Overhead."
- Duration: 35 minutes during Sunday online session.

Review

Samar reviews hash calculation performance; Tejas reviews audit UI rendering; Nidhish verifies payload determinism.

Documentation

Document the audit ledger mathematical model, schema, and tampering verification script in `docs/05-system-design/audit-ledger-design.md` and `developer_plan.md`.

Expected Result

A fully functional, mathematically verifiable audit ledger in PostgreSQL capable of detecting any database manipulation.

Definition of Done

`test_audit_tampering.py` successfully validates an untampered ledger and flags any modified row with 100% accuracy on all 6 laptops.

WEEK 04

Local LLM Runtime, Model Registry & Adaptive Router

FRIDAY

Day 10

Friday — Day 10

Goal

Configure the production local Ollama multi-model runtime, establish local model pulling scripts, and benchmark inference latency and VRAM footprint on developer hardware.

Task Sequence

- DEV-010.1 [Nidhish]: Standardize local model portfolio: qwen2.5:14b (reasoning/standards), deepseek-r1:14b or deepseek-r1:8b (math/code), and qwen2.5-vl:7b (vision).
- DEV-010.2 [Nidhish]: Write automated model verification script `scripts/pull_models.sh` that checks available local models and pulls missing weights.
- DEV-010.3 [Samar]: Implement asynchronous Ollama client wrapper in `backend/app/core/llm.py` with keep-alive management and connection pooling.
- DEV-010.4 [Dakshit]: Test vision model loading (qwen2.5-vl:7b); measure VRAM impact when processing high-resolution test inspection images.
- DEV-010.5 [Shreya]: Audit Ollama network bindings; verify that Ollama listens strictly on `127.0.0.1:11434` and rejects external network requests.
- DEV-010.6 [Tejas]: Test Ollama streaming API; ensure chunk-by-chunk token emission can be streamed over SSE to the frontend with low latency.
- DEV-010.7 [Safa]: Run benchmark scripts across all 6 developer machines; document tokens/sec and memory footprints in a comparison table.

Developer Responsibilities (In Execution Order)

- Nidhish (Model Portfolio Lead): Selects optimal model quantizations (Q4KM), creates `scripts/pull_models.sh`, and implements `<think>...</think>` tag stripping.
- Samar (Async LLM Client Architect): Writes the asynchronous client in `backend/app/core/llm.py`, handling connection drops and timeouts gracefully.
- Dakshit (Vision Model Benchmark Specialist): Tests multi-modal weight initialization and measures memory spikes during image processing.
- Shreya (Air-Gap Network Auditor): Verifies that local model execution generates 0 external network requests and checks that telemetry reporting in Ollama is disabled.
- Tejas (Token Stream Integrator): Connects the backend token yield generator to the frontend message stream, measuring rendering latency.
- Safa (Hardware Benchmark Analyst): Executes standardized benchmark queries on all 6 laptops and compiles the comparative hardware performance matrix.

Collaboration Flow

Nidhish and Samar configure the runtime and async client; Dakshit tests vision models; Shreya validates network security; Tejas tests streaming; Safa compiles performance benchmarks.

Prerequisites

- Week 3 completed.
- Ollama installed and running locally.
- Disk space $\geq$ 15GB for model weights.

Hands-On Requirement

All 6 developers must execute `scripts/pull_models.sh`, pull the assigned local model, and execute `python scripts/benchmark_llm.py` to record their machine's tokens/second.

Knowledge Sharing

- Presenter: Nidhish
- Topic: "Open-Weight Industrial SLMs: Quantization (Q4KM vs Q8), VRAM Budgets & Why Local Models Outperform Cloud APIs in Specialized Domains."
- Duration: 25 minutes during Friday offline sync.

Review

Samar reviews async client connection management; Shreya verifies loopback network binding.

Documentation

Publish hardware benchmark table and model pulling guide in `docs/07-ai-ml/model-registry.md` and `developer_plan.md`.

Expected Result

Verified local Ollama runtime operating on all 6 machines with zero internet access required during inference.

Definition of Done

`python backend/app/core/llm.py` successfully connects to local Ollama, queries the model, and prints response with zero external network traffic.

SATURDAY

Day 11

Saturday — Day 11

Goal

Implement the deterministic, explainable Adaptive AI Model Router (ADR-001), dynamically routing user queries based on modality, complexity, VRAM headroom, and latency targets.

Task Sequence

1. DEV-011.1 [Nidhish]: Build query classification heuristic in `backend/app/agent/router.py` (categorizing tasks into: `VISUAL_INSPECTION`, `CORROSION_MATH`, `SOP_RETRIEVAL`, `GENERAL_QUERY`).

2. DEV-011.2 [Nidhish]: Implement multi-factor routing scoring function:

$$\text{Score}(M, T) = w_1 \cdot \text{ModalityMatch} + w_2 \cdot \text{ComplexityFit} + w_3 \cdot \text{VRAMHeadroom} - w_4 \cdot \text{Latency}$$

3. DEV-011.3 [Samar]: Integrate router into task pipeline; verify router execution completes in $< 50 \text{ ms}$ without blocking the async event loop.
4. DEV-011.4 [Dakshit]: Formulate test prompts for multimodal inspection tasks (P&ID diagrams, corrosion photos) to verify routing to Qwen-VL.
5. DEV-011.5 [Safa]: Author test prompts for regulatory compliance (OISD, API 570) and verify routing to Qwen2.5:14b.
6. DEV-011.6 [Shreya]: Test edge cases: simulate Ollama VRAM exhaustion and verify router automatically downgrades to the lighter fallback model.

7. DEV-011.7 [Tejas]: Update `ModelRouterCard.tsx` in frontend to render the router's explainable rationale, VRAM meter, and selected model badge.

Developer Responsibilities (In Execution Order)

1. Nidhish (Routing Algorithm Architect): Implements the deterministic heuristic classifier and multi-factor scoring algorithm in `backend/app/agent/router.py`.
2. Samar (Pipeline Integration Lead): Connects the router output into the task pipeline, ensuring structured decisions are emitted over SSE.
3. Dakshit (Multimodal Test Engineer): Crafts test suites for image-based and schematic queries, verifying deterministic routing to vision models.
4. Safa (Standards Query Test Engineer): Crafts test suites for complex regulatory queries, verifying routing to high-parameter reasoning models.
5. Shreya (Resilience & Fallback Tester): Simulates low VRAM conditions, verifying that the router degrades safely to lightweight quantized models.
6. Tejas (Router UI Visualizer): Updates the frontend `ModelRouterCard` to present the selection rationale clearly to the operator.

Collaboration Flow

Nidhish writes the core heuristic; Samar integrates it into the pipeline; Dakshit and Safa feed domain test prompts; Shreya tests VRAM exhaustion edge cases; Tejas visualizes the decision in the UI.

Prerequisites

- DEV-010 completed.
- Understanding of ADR-001.

Hands-On Requirement

Every developer runs `pytest backend/tests/test_router.py` and inspects the terminal output showing 20 distinct prompts routing to their optimal models with explainable rationale.

Knowledge Sharing

- Presenter: Nidhish & Dakshit
- Topic: "Deterministic Heuristic Routing vs. Contextual Bandits: Why Predictability and Explainability are Mandatory for Safety-Critical Operations."
- Duration: 30 minutes during Saturday online session.

Review

Samar reviews router latency and algorithmic complexity; Shreya verifies safe degradation when VRAM is saturated.

Documentation

Document routing decision matrix and scoring weights in `docs/04-architecture/adaptive-router.md` and `developer_plan.md`.

Expected Result

A deterministic, explainable router selecting the optimal local model in `<50ms` with full machine-readable justification.

Definition of Done

`test_router.py` achieves 100% pass rate across all 20 test prompts; frontend renders the routing decision card with live rationale.

SUNDAY

Day 12

Sunday — Day 12

Goal

Implement the Constitutional Policy Layer (ADR-002) enforcing tiered L0-L4 safety boundaries, prompt injection sanitization, and structured JSON schema extraction.

Task Sequence

1. DEV-012.1 [Nidhish & Shreya]: Implement L0-L4 policy engine in `backend/app/core/policy.py`:
 - L0: Immutable system baseline (refinery safety invariants).
 - L1: Role-based boundary enforcement (operator vs lead engineer permissions).
 - L2: Operational context restrictions (unit-specific operating limits).
 - L3: Grounding & citation mandate (unsupported claims strictly prohibited).
 - L4: Risk-triggered human sign-off directives.
2. DEV-012.2 [Dakshit]: Author adversarial test cases: test jailbreaks, prompt injection attacks, and delimiter collision attempts.
3. DEV-012.3 [Nidhish]: Implement prompt injection filter scanning incoming prompts for jailbreaks, prompt leaks, and role overrides.
4. DEV-012.4 [Samar]: Implement structured JSON extraction utility (`extract_json_from_text`) handling markdown fences, malformed braces, and trailing commas.
5. DEV-012.5 [Safa]: Verify policy violation logging; ensure every rejected prompt generates a high-severity entry in PostgreSQL `audit_logs`.
6. DEV-012.6 [Tejas]: Implement policy warning banners in frontend `ChatContainer.tsx` displaying policy violation notices when prompts are blocked.

Developer Responsibilities (In Execution Order)

1. Nidhish (Constitutional Engine Lead): Authors the system prompt wrappers and L0-L4 policy rules in `backend/app/core/policy.py`.
2. Shreya (Industrial Safety Compliance Officer): Aligns the constitutional invariants with OISD-105 standards, ensuring physical operating limits cannot be overridden.
3. Dakshit (Adversarial Red-Teamer): Attempts prompt injection attacks, role hijacking, and delimiter fuzzing against the policy engine.
4. Samar (JSON Parser Engineer): Writes the resilient JSON extraction parser, recovering structured outputs from noisy LLM responses.
5. Safa (Violation Audit Specialist): Verifies that intercepted attacks are immediately recorded with high severity in the database audit log.
6. Tejas (Policy Alert UI Engineer): Builds alert modals and warning banners in the frontend when an operator submits a blocked prompt.

Collaboration Flow

Nidhish and Shreya build the policy rules; Dakshit stress tests with adversarial attacks; Samar implements the JSON parser; Safa verifies audit logging; Tejas builds UI alerts.

Prerequisites

- DEV-010 and DEV-011 completed.
- Understanding of ADR-002.

Hands-On Requirement

All 6 developers must run `pytest backend/tests/test_policy.py`, verify all injection attacks are intercepted, and test the JSON extractor against malformed LLM outputs.

Knowledge Sharing

- Presenter: Shreya & Nidhish
- Topic: "Constitutional AI & Guardrail Engineering: Preventing Prompt Injections and Hallucinations in Hazardous Environments."
- Duration: 35 minutes during Sunday online session.

Review

Samar reviews JSON parser regex performance; Shreya verifies policy immutability.

Documentation

Document the complete L0-L4 policy specification and prompt templates in `docs/04-architecture/constitutional-policy.md` and `developer_plan.md`.

Expected Result

Robust constitutional guardrails that intercept adversarial prompts and reliably extract clean JSON data from local LLM outputs.

Definition of Done

Policy engine blocks 100% of injection test cases; JSON extractor recovers valid data from 10 malformed test outputs; audit log records all violations.

WEEK 05

Multimodal Ingestion Engine & High-Fidelity OCR

FRIDAY

Day 13

Friday — Day 13

Goal

Build the universal file upload and document parsing engine, supporting secure local storage, MIME magic-byte validation, and multi-format extraction (PDF, DOCX, XLSX, images).

Task Sequence

- DEV-013.1 [Samar]: Implement secure file ingestion endpoint `POST /api/upload` storing files in segregated directory `/data/uploads/` with UUID prefixes.
- DEV-013.2 [Shreya]: Implement MIME type magic-byte inspection using `python-magic` to prevent extension spoofing (e.g. `.exe` disguised as `.pdf`).
- DEV-013.3 [Dakshit]: Build modular document extractors in `backend/app/core/file_extractor.py` (PyPDF for text, `python-docx` for tables, `openpyxl` for spreadsheets).
- DEV-013.4 [Nidhish]: Test extracted text feeds into LLM prompt contexts; verify truncation safety for documents exceeding token context limits.
- DEV-013.5 [Tejas]: Build file upload dropzone in frontend `Composer.tsx` with drag-and-drop, upload progress bar, and file type validation.
- DEV-013.6 [Safa]: Collect and curate real-world industrial test files (scanned NDT logs, API standard excerpts, sample inspection spreadsheets); verify database persistence.

Developer Responsibilities (In Execution Order)

- Samar (Upload Gateway Engineer): Implements `POST /api/upload`, filesystem storage paths, and metadata persistence in PostgreSQL.
- Shreya (Upload Security Inspector): Implements MIME magic-byte validation, path traversal defense, and 25MB file size ceilings.
- Dakshit (Document Extraction Specialist): Builds the core extraction logic for PDF, Word, and Excel files in `backend/app/core/file_extractor.py`.
- Nidhish (Context Budget Specialist): Verifies that extracted document text is cleanly segmented and fits within the model's token context window.
- Tejas (Upload Interface Lead): Builds the drag-and-drop file upload zone in `Composer.tsx` with live progress bars and file previews.
- Safa (Test Corpus Lead & Verifier): Uploads test files across all formats, verifies database metadata records, and logs extraction accuracy.

Collaboration Flow

Samar and Shreya secure the upload endpoint; Dakshit implements format extractors; Nidhish tests context chunking; Tejas builds the UI dropzone; Safa verifies the real test files.

Prerequisites

- Week 4 completed.

- Sample PDF, DOCX, and XLSX files placed in data/sample_reports/.

Hands-On Requirement

All 6 developers must upload a sample NDT report via their local frontend, verify it appears in data/uploads/, inspect the extracted text in the database, and verify the SHA-256 hash.

Knowledge Sharing

- Presenter: Dakshit
- Topic: "Industrial Document Ingestion: Handling Corrupted PDFs, Non-Standard Fonts & Preventing Malicious File Payloads."
- Duration: 25 minutes during Friday offline sync.

Review

Shreya reviews file validation security; Samar reviews database record creation; Tejas tests upload UI error handling.

Documentation

Document supported formats, size limits, and extractor architectures in docs/07-ai-ml/multimodal-pipeline.md and developer_plan.md.

Expected Result

A secure upload pipeline that validates, stores, and extracts structured text and metadata from industrial documents into PostgreSQL.

Definition of Done

POST /api/upload successfully parses PDF, DOCX, and XLSX test files, rejects spoofed extensions, and stores metadata in PostgreSQL with 100% test pass rate.

SATURDAY

Day 14

Saturday — Day 14

Goal

Integrate PaddleOCR into the backend pipeline to accurately extract tabular wall-thickness measurement logs from scanned, degraded physical inspection sheets.

Task Sequence

1. DEV-014.1 [Dakshit]: Install and configure PaddleOCR and PaddlePaddle dependencies in the backend environment.
2. DEV-014.2 [Dakshit]: Build table extraction tool in backend/app/tools/ocr_tool.py optimized for tabular NDT layouts (Point ID, Nominal, Actual, Minimum Required).
3. DEV-014.3 [Samar]: Implement async thread-pool worker for OCR processing to prevent CPU-intensive computer vision tasks from freezing the FastAPI event loop.
4. DEV-014.4 [Nidhish]: Connect OCR output to the prompt context; verify that structured tables format cleanly as Markdown tables for LLM reasoning.
5. DEV-014.5 [Shreya]: Verify that PaddleOCR models execute 100% locally with zero external network calls to third-party model hubs during inference.
6. DEV-014.6 [Tejas]: Build tabular data preview modal in frontend allowing operators to inspect and verify OCR-extracted tables before agent processing.
7. DEV-014.7 [Safa]: Test OCR accuracy across 5 different scanned inspection logs; calculate character and numerical error rates on thickness data.

Developer Responsibilities (In Execution Order)

1. Dakshit (OCR Engineering Lead): Implements PaddleOCR integration, bounding-box reconstruction, and column parsing for tabular NDT reports.
2. Samar (Async Execution Specialist): Wraps OCR processing in an async background executor, ensuring the server handles concurrent requests without freezing.
3. Nidhish (Context Formatter): Converts raw OCR bounding-box dictionaries into clean markdown tables formatted for LLM reasoning.
4. Shreya (Air-Gap Compliance Auditor): Confirms that PaddleOCR model weights are loaded strictly from local directories with zero telemetry.
5. Tejas (Table Preview UI Engineer): Builds the interactive modal allowing operators to view and edit OCR-extracted tables before submission.
6. Safa (OCR Accuracy Evaluator): Benchmarks OCR accuracy against 5 real-world scanned sheets, measuring numerical precision.

Collaboration Flow

Dakshit leads PaddleOCR integration; Samar manages thread-pool concurrency; Nidhish formats the tables; Shreya verifies offline execution; Tejas builds the preview UI; Safa evaluates accuracy.

Prerequisites

- DEV-013 completed.
- Python `paddleocr` and dependencies installed.

Hands-On Requirement

Every developer runs `python -m pytest backend/tests/test_ocr.py`, passing a scanned sample NDT image, and verifies that the output JSON correctly extracts the thickness measurement values.

Knowledge Sharing

- Presenter: Dakshit & Safa
- Topic: "Optical Character Recognition in Industrial Plants: Extracting Tabular Data from Degraded and Coffee-Stained Inspection Sheets."
- Duration: 30 minutes during Saturday online session.

Review

Dakshit and Samar review OCR performance and memory utilization; Shreya verifies local-only model weight loading.

Documentation

Document PaddleOCR architecture, pre-processing filters, and output JSON schema in `docs/07-ai-ml/multimodal-pipeline.md` and `developer_plan.md`.

Expected Result

High-precision tabular extraction capable of converting scanned PDF inspection sheets into structured, machine-readable JSON tables.

Definition of Done

`test_ocr.py` successfully extracts all measurement rows from the test NDT log with $>95\%$ numerical accuracy on all 6 laptops.

SUNDAY

Day 15

Sunday — Day 15

Goal

Integrate Qwen2.5-VL vision-language reasoning for physical defect inspection and P&ID schematic understanding, completing the multimodal analysis engine.

Task Sequence

1. DEV-015.1 [Nidhish]: Configure qwen2.5-vl:7b in Ollama; build multimodal query helper in backend/app/core/llm.py supporting base64-encoded image payloads.
2. DEV-015.2 [Dakshit & Nidhish]: Build visual defect inspection tool (backend/app/tools/vision_tool.py) detecting pitting corrosion, flange leaks, and weld defects from photographs.
3. DEV-015.3 [Samar]: Manage memory during dual OCR and VLM execution; ensure image buffers are cleared from RAM immediately following inference.
4. DEV-015.4 [Shreya]: Verify that high-resolution plant photographs are never stored in temporary public directories or exposed to insecure endpoints.
5. DEV-015.5 [Tejas]: Update frontend chat message renderer (MessageBubble.tsx) to display uploaded images alongside visual inspection findings and bounding boxes.
6. DEV-015.6 [Safa]: Run visual tests on 3 distinct defect photos (pitting corrosion, valve corrosion, weld crack); record model descriptions in test report.

Developer Responsibilities (In Execution Order)

1. Nidhish (VLM Integration Lead): Builds base64 image encoding and multi-modal request formatting for Ollama's API.
2. Dakshit (Visual Prompt Engineer): Authors specialized industrial prompts for identifying corrosion types and reading P&ID symbols.
3. Samar (Memory & Buffer Manager): Implements immediate memory cleanup for high-resolution image tensors post-inference.
4. Shreya (Image Asset Security Auditor): Audits image storage directories, verifying permissions and path isolation.
5. Tejas (Visual Telemetry UI Lead): Implements image rendering, zoom modals, and bounding-box overlays in the chat interface.
6. Safa (VLM Test & Verification Specialist): Tests defect recognition across three different corrosion images and logs accuracy metrics.

Collaboration Flow

Nidhish and Dakshit code the VLM integration; Samar manages memory buffers; Shreya audits file access; Tejas renders visual outputs in the UI; Safa validates defect detection.

Prerequisites

- DEV-013 and DEV-014 completed.
- qwen2.5-vl:7b downloaded locally in Ollama.

Hands-On Requirement

All 6 developers must execute `python scripts/test_vision_pipeline.py`, passing `corrosion_flange.png`, and observe the local VLM identify localized pitting corrosion near the weld zone.

Knowledge Sharing

- Presenter: Nidhish & Dakshit
- Topic: "Vision-Language Models (VLMs) in Critical Infrastructure: Detecting Mechanical Degradation and Reading Complex P&ID Diagrams."
- Duration: 35 minutes during Sunday online session.

Review

Samar reviews memory cleanup; Tejas reviews image rendering in UI; Shreya audits file access permissions.

Documentation

Document the multimodal pipeline, prompt structures, and visual test results in `docs/07-ai-ml/multimodal-pipeline.md` and `developer_plan.md`.

Expected Result

A multimodal engine capable of analyzing both tabular text logs (PaddleOCR) and physical defect photographs (Qwen2.5-VL) locally.

Definition of Done

Automated test passes where an uploaded image and PDF produce a unified JSON analysis containing both tabular metrics and visual defect descriptions.

WEEK 06

Sovereign Vector Retrieval & Hybrid Agentic RAG

FRIDAY

Day 16

Friday — Day 16

Goal

Deploy the Qdrant Rust-based vector search engine locally on port 6333, configure local BGE-M3 dense embeddings, and create sovereign vector collections.

Task Sequence

- DEV-016.1 [Samar]: Deploy Qdrant vector database service locally (native binary or Docker container) exposing port 6333 with persistent local storage.
- DEV-016.2 [Dakshit]: Implement local embedding generator in `backend/app/models/embeddings.py` using BAAI/bge-m3 (1024-dimensional dense vectors) running strictly on CPU/GPU locally.
- DEV-016.3 [Samar & Dakshit]: Create initialized Qdrant collection `sovereign_sops` with cosine distance metric and HNSW indexing parameters.
- DEV-016.4 [Shreya]: Verify zero egress for embedding models: ensure bge-m3 weights are cached locally in `/data/models/` and do not attempt HuggingFace downloads.
- DEV-016.5 [Tejas]: Add Qdrant service status indicator to frontend `Header.tsx` and `ContextPanel.tsx` reflecting vector database connectivity.
- DEV-016.6 [Safa]: Test Qdrant connectivity across all 6 machines; verify that Qdrant REST API (`http://localhost:6333/dashboard`) is accessible locally.

Developer Responsibilities (In Execution Order)

- Samar (Vector Engine Deployer): Starts the Qdrant service on port 6333, configures persistent volume mounts, and builds the connection client.
- Dakshit (Dense Embeddings Lead): Integrates the BGE-M3 model, benchmark embedding latency per token chunk, and handles vector serialization.
- Shreya (Offline Weights Auditor): Audits model caching, verifying that embedding vectors are calculated locally with no internet connection.
- Nidhish (Chunking Strategy Specialist): Formulates chunking boundaries (512 tokens) preserving regulatory section numbers and headers.
- Tejas (Service Telemetry Integrator): Adds Qdrant health indicators and collection status meters to the frontend header.
- Safa (Vector Store Test Engineer): Executes verification scripts across all developer machines, testing collection initialization.

Collaboration Flow

Samar and Dakshit deploy Qdrant and embeddings; Shreya validates offline isolation; Nidhish optimizes chunking; Tejas updates UI telemetry; Safa verifies multi-machine connectivity.

Prerequisites

- Week 5 completed.

- Qdrant binary or Docker available.
- Port 6333 free.

Hands-On Requirement

Every developer must start Qdrant, run `python backend/app/rag/vector_store.py`, and observe the script create the `sovereign_sops` collection and insert a test vector.

Knowledge Sharing

- Presenter: Samar & Dakshit
- Topic: "Vector Databases & HNSW Indexing: Why Qdrant and Dense Embeddings Enable Millisecond Retrieval Across Massive Document Repositories."
- Duration: 25 minutes during Friday offline sync.

Review

Shreya audits offline model caching; Samar reviews connection pooling and error recovery.

Documentation

Document Qdrant deployment, collection schema, and embedding parameters in `docs/07-ai-ml/hybrid-rag.md` and `developer_plan.md`.

Expected Result

Local Qdrant vector engine operational on all 6 machines, generating 1024-dimensional BGE-M3 embeddings without external connectivity.

Definition of Done

`curl http://localhost:6333/collections/sovereign_sops` returns `status: green` on all 6 laptops with test vector successfully stored and retrieved.

SATURDAY

Day 17

Saturday — Day 17

Goal

Ingest, chunk, embed, and index synthetic industrial regulatory standards and MRPL Standard Operating Procedures into Qdrant with tagged metadata collections.

Task Sequence

1. DEV-017.1 [Nidhish]: Curate and format synthetic industrial SOP corpus in `data/sample_sops/` (API 570, API 510, API 653, OISD-105).
2. DEV-017.2 [Dakshit]: Implement hierarchical document chunker preserving chapter, section, and page metadata in `backend/app/rag/indexer.py`.
3. DEV-017.3 [Samar]: Implement batch ingestion CLI pipeline reading SOP files, generating embeddings, and uploading points to Qdrant with retry logic.
4. DEV-017.4 [Shreya]: Verify vector payload security; ensure no proprietary plant credentials or unredacted personnel names exist in sample SOPs.
5. DEV-017.5 [Tejas]: Build `#Collection` selector dropdown in frontend `Composer.tsx` allowing operators to scope retrieval to specific units (e.g. `#Hydrocracker`).
6. DEV-017.6 [Safa]: Execute the ingestion script; verify vector point counts in Qdrant web dashboard; validate metadata tags.

Developer Responsibilities (In Execution Order)

1. Nidhish (SOP Corpus Curator): Structures realistic engineering standards with precise technical clauses, tables, and formula definitions.
2. Dakshit (Hierarchical Chunker Engineer): Implements document chunking preserving parent-child section relationships and page tags.
3. Samar (Batch Indexing Pipeline Lead): Builds the high-throughput batch indexer, writing points to Qdrant with metadata payloads.
4. Shreya (Data Sanitation Auditor): Reviews indexed payloads to ensure clean regulatory text with zero proprietary secrets.
5. Tejas (Collection Selector UI Lead): Adds the #Collection tagging filter to the prompt composer in the frontend.
6. Safa (Index Verification Specialist): Executes the batch indexing CLI, inspects vector points in the Qdrant dashboard, and records performance.

Collaboration Flow

Nidhish curates documents; Dakshit and Samar build the batch indexing engine; Shreya audits data; Tejas builds collection selectors; Safa verifies indexing.

Prerequisites

- DEV-016 completed.
- Sample SOP documents prepared in `data/sample_sops/`.

Hands-On Requirement

All 6 developers execute `python -m app.rag.indexer` on their machines, watch the batch progress bar, and verify in Qdrant dashboard that 500+ vectors are indexed.

Knowledge Sharing

- Presenter: Dakshit & Nidhish
- Topic: "Industrial Knowledge Ingestion: Chunking Strategies, Header Preservation & Metadata Filtering for High-Precision Engineering RAG."
- Duration: 30 minutes during Saturday online session.

Review

Samar reviews indexing batch throughput; Nidhish reviews chunk boundary coherence; Shreya audits metadata integrity.

Documentation

Document SOP corpus inventory, chunking rules, and ingestion CLI commands in `docs/06-data/corpus-management.md` and `developer_plan.md`.

Expected Result

Fully indexed industrial SOP knowledge base in Qdrant with complete chapter, section, and tag metadata.

Definition of Done

Qdrant collection contains all synthetic SOP vectors with verified payload fields; test query retrieves matching document points.

SUNDAY

Day 18

Sunday — Day 18

Goal

Implement the Hybrid Agentic RAG Retriever, combining dense semantic vector search with keyword/tag filtering, and displaying interactive citation cards in the frontend.

Task Sequence

1. DEV-018.1 [Dakshit]: Implement hybrid retriever in `backend/app/rag/retriever.py` querying Qdrant with semantic vector similarity and payload filters.
2. DEV-018.2 [Samar]: Benchmark retriever latency; ensure search, filtering, and formatting complete in $<150\text{ms}$.
3. DEV-018.3 [Nidhish]: Integrate retrieved chunks into the LLM context prompt; implement prompt delimiters (`<context>...</context>`) preventing hallucinated citations.
4. DEV-018.4 [Shreya]: Verify that retriever strictly respects collection permission tags, preventing unauthorized document retrieval across segregated units.
5. DEV-018.5 [Tejas]: Build and polish `RAGSourceCard.tsx` in frontend with expandable excerpt view, similarity score badge, and source page link.
6. DEV-018.6 [Safa]: Author and execute 15 standard industrial benchmark queries; record Top-1 and Top-3 accuracy metrics in testing logs.

Developer Responsibilities (In Execution Order)

1. Dakshit (Retriever Algorithm Lead): Implements vector search scoring, payload tag matching, and cosine similarity thresholding (≥ 0.70).
2. Samar (Query Optimizer): Optimizes Qdrant client connection pooling, keeping retrieval times strictly under 150ms.
3. Nidhish (Context Prompt Assembler): Formats retrieved clauses into the model prompt, instructing the LLM to cite specific clauses.
4. Shreya (Access Isolation Auditor): Verifies that scoped queries cannot access documents from unselected plant collections.
5. Tejas (Citation Card UI Lead): Renders interactive citation cards in the chat feed with direct page links and text snippets.
6. Safa (Benchmark Test Lead): Executes the 15-query test suite, validating that API 570 Table 4 is returned as Top-1 for piping queries.

Collaboration Flow

Dakshit and Samar build the retrieval logic; Nidhish formats the context; Shreya verifies isolation; Tejas renders citation cards; Safa measures accuracy.

Prerequisites

- DEV-016 and DEV-017 completed.
- Qdrant collection seeded with SOP vectors.

Hands-On Requirement

All 6 developers run `python scripts/test_rag_retrieval.py`, observe the Top-3 retrieved clauses for an NDT inspection query, and verify similarity scores exceed 0.82 .

Knowledge Sharing

- Presenter: Dakshit & Tejas
- Topic: "Grounding AI in Authoritative Engineering Truth: Vector Similarity, Threshold Gating & Interactive UI Citations."
- Duration: 35 minutes during Sunday online session.

Review

Nidhish reviews citation fidelity; Samar reviews async query efficiency; Shreya verifies tag-based isolation.

Documentation

Document retriever scoring algorithm, threshold parameters, and citation specifications in `docs/07-ai-ml/hybrid-rag.md` and `developer_plan.md`.

Expected Result

A high-precision hybrid RAG retriever returning relevant regulatory clauses with page citations in $<150\text{ms}$, displayed interactively in the UI.

Definition of Done

Automated test achieves $>90\%$ Top-3 accuracy across 15 industrial test queries; frontend renders clickable citation cards with verified text excerpts.

WEEK 07

LangGraph Agentic Orchestration & Cyclic State Machine

FRIDAY

Day 19

Friday — Day 19

Goal

Scaffold and compile the LangGraph StateGraph (ADR-003), defining the node topology, edge transitions, conditional routing, and error fallback channels.

Task Sequence

- DEV-019.1 [Samar]: Install langgraph in the backend environment; create module backend/app/agent/.
- DEV-019.2 [Samar]: Define state graph topology in backend/app/agent/graph.py with nodes:
ingest \$to\$ route \$to\$ extract \$to\$ retrieve \$to\$ plan \$to\$ sandbox \$to\$ critique \$to\$ human_checkpoint \$to\$ synthesize.
- DEV-019.3 [Nidhish]: Define routing node edge conditions; ensure router output directs execution to the appropriate specialized sub-graphs.
- DEV-019.4 [Shreya]: Review graph termination guarantees: ensure cyclic loops are bounded by hard counters (retry_count <= 2) to prevent infinite execution loops.
- DEV-019.5 [Dakshit]: Review graph transitions for multimodal data flow: ensure image and document payloads pass cleanly from ingest to extract.
- DEV-019.6 [Tejas]: Align frontend ExecutionTimeline.tsx with exact LangGraph node names to ensure 1-to-1 visual synchronization of active steps.
- DEV-019.7 [Safa]: Generate visual Mermaid diagram of the compiled graph using LangGraph's drawing utility; publish diagram to documentation.

Developer Responsibilities (In Execution Order)

- Samar (LangGraph Architect): Builds the core StateGraph(WorkbenchState) and compiles the graph topology in backend/app/agent/graph.py.
- Nidhish (Conditional Edge Designer): Configures routing conditions and critique evaluation branch decisions.
- Shreya (Loop Termination & Safety Guard): Enforces hard cycle limits (max_retries=2) to guarantee the state machine never enters infinite loops.
- Dakshit (Multimodal State Flow Inspector): Traces binary image and document payloads through the graph nodes to prevent state bloat.
- Tejas (UI Stepper Synchronizer): Updates frontend step definitions in ExecutionTimeline.tsx to match graph node names exactly.
- Safa (State Machine Visualizer & Scribe): Generates and publishes the official Mermaid state diagram and documentation.

Collaboration Flow

Samar builds the state graph; Nidhish sets up edge branching; Shreya adds loop guards; Dakshit checks data flow; Tejas aligns the UI timeline; Safa produces the diagram.

Prerequisites

- Week 6 completed.
- langgraph installed in backend/requirements.txt.

Hands-On Requirement

Every developer runs `python -c "from app.agent.graph import compile_graph; g = compile_graph(); print('Graph compiled successfully!')"` and inspects the generated Mermaid state diagram.

Knowledge Sharing

- Presenter: Samar
- Topic: "Cyclic State Machines vs Linear Chains: Why LangGraph Solves the Fragility of Traditional LLM Pipelines."
- Duration: 25 minutes during Friday offline sync.

Review

Shreya verifies infinite-loop guards; Samar reviews state persistence across nodes; Tejas confirms node names match frontend timeline.

Documentation

Document the complete graph topology, state schema, and Mermaid visual flowchart in `docs/05-system-design/orchestrator-langgraph.md` and `developer_plan.md`.

Expected Result

A compiled, validated LangGraph cyclic state machine enforcing bounded execution loops and robust error channels.

Definition of Done

`graph.compile()` executes without schema warnings, passes cyclic boundary tests, and generates a valid state diagram on all 6 laptops.

SATURDAY

Day 20

Saturday — Day 20

Goal

Implement individual specialized graph node functions (`ingest_node`, `route_node`, `retrieve_node`, `plan_node`) and wire them into the live `StateGraph`.

Task Sequence

1. DEV-020.1 [Dakshit]: Implement `ingest_node`: validates task payload, retrieves uploaded files, and initializes `WorkbenchState`.
2. DEV-020.2 [Samar]: Implement `route_node`: invokes the Adaptive Model Router and records explainable model selection in state.
3. DEV-020.3 [Dakshit]: Implement `retrieve_node`: executes hybrid RAG search against Qdrant and appends grounded SOP citations to state.
4. DEV-020.4 [Nidhish]: Implement `plan_node`: formats engineering context and prompts the selected LLM to formulate an inspection analysis plan.
5. DEV-020.5 [Shreya]: Audit data passed between nodes; verify that sensitive file paths or credentials are not leaked into LLM prompt text.
6. DEV-020.6 [Tejas]: Verify that each node execution dispatches an SSE event (`emit_step`) displaying node progress and intermediate output in the UI.
7. DEV-020.7 [Safa]: Author comprehensive mock states for unit testing; verify that node failures emit proper error state dictionaries.

Developer Responsibilities (In Execution Order)

1. Dakshit (Ingest & Retrieve Node Engineer): Writes `ingest_node` and `retrieve_node`, integrating file extractors and Qdrant queries into graph state.
2. Samar (Route Node Engineer): Writes `route_node`, binding the Adaptive Model Router to the graph state.
3. Nidhish (Plan Node Engineer): Writes `plan_node`, assembling prompts for LLM reasoning and generating structured engineering plans.
4. Shreya (Inter-Node Security Auditor): Audits state dictionaries between transitions, verifying that internal file paths are sanitized.
5. Tejas (Step Emission Integrator): Connects node transitions to live SSE broadcasting, displaying step cards in the UI.
6. Safa (Node Unit Test Lead): Authors `backend/tests/test_agent_nodes.py` and verifies all nodes execute cleanly in isolation.

Collaboration Flow

Dakshit, Samar, and Nidhish implement specialized nodes; Shreya reviews data security; Tejas verifies step event emission; Safa leads unit testing.

Prerequisites

- DEV-019 completed.
- All previous subsystems (Router, RAG, Ingestion) operational.

Hands-On Requirement

All 6 developers execute `pytest backend/tests/test_agent_nodes.py`, verifying that each individual node executes, mutates state correctly, and passes validation.

Knowledge Sharing

- Presenter: Samar & Nidhish
- Topic: "Designing Isolated Agent Nodes: Functional State Transformation, Pure Functions & State Immutability."
- Duration: 30 minutes during Saturday online session.

Review

Dakshit reviews retrieval node queries; Nidhish reviews prompt assembly; Samar validates state mutation rules.

Documentation

Document node specifications, inputs, outputs, and side-effects in `docs/05-system-design/orchestrator-langgraph.md` and `developer_plan.md`.

Expected Result

Four fully operational core LangGraph nodes executing in sequence and transforming `WorkbenchState` deterministically.

Definition of Done

`test_agent_nodes.py` passes 100% across all 4 nodes with verified state transformations and SSE event emissions.

SUNDAY

Day 21

Sunday — Day 21

Goal

Refactor the FastAPI `/api/chat` route to execute the compiled LangGraph state machine asynchronously in the background, replacing the legacy procedural fallback pipeline.

Task Sequence

1. DEV-021.1 [Samar]: Refactor backend/app/api/chat.py to initialize and invoke the compiled LangGraph graph using `graph.astream()`.
2. DEV-021.2 [Samar]: Implement background task dispatch: return 200 OK with `task_id` immediately to client while graph executes asynchronously.
3. DEV-021.3 [Tejas]: Test frontend chat stream handling; verify that real-time tokens and step transitions update live in the UI without browser stutter.
4. DEV-021.4 [Dakshit]: Verify that document context uploaded in UI passes cleanly through FastAPI into the background graph execution.
5. DEV-021.5 [Nidhish]: Monitor Ollama token generation during graph execution; ensure long inferences do not trigger Unicorn worker timeouts.
6. DEV-021.6 [Shreya]: Monitor socket and thread states during async execution; verify background tasks clean up properly upon completion or client disconnect.
7. DEV-021.7 [Safa]: Execute load test: trigger 3 concurrent chat tasks; verify database records all steps with correct foreign keys.

Developer Responsibilities (In Execution Order)

1. Samar (Async Execution Architect): Replaces the legacy procedural chat runner with `graph.astream()`, broadcasting step events to the SSE queue.
2. Tejas (Stream Consumer Integrator): Verifies that the frontend receives and displays real-time state transitions without UI freezing.
3. Dakshit (Context Pipeline Tester): Confirms that file attachments uploaded via the UI are correctly fed into the background graph worker.
4. Nidhish (Inference Timeout Specialist): Tunes Ollama timeout parameters to ensure complex multi-node reasoning completes safely.
5. Shreya (Resource & Socket Monitor): Inspects thread and memory usage during background execution, ensuring zero socket leaks.
6. Safa (Concurrency & Persistence Tester): Executes concurrent tasks and verifies that all node executions persist cleanly in PostgreSQL `agent_steps`.

Collaboration Flow

Samar leads backend refactoring; Tejas synchronizes client stream handling; Dakshit and Nidhish verify context and LLM inference; Shreya monitors server health; Safa executes concurrency tests.

Prerequisites

- DEV-019 and DEV-020 completed.
- FastAPI backend and Next.js frontend running.

Hands-On Requirement

All 6 developers must open `http://localhost:3000`, type an industrial prompt ("Analyze pipe HC-102-B"), submit, and watch the real-time LangGraph nodes illuminate sequentially on their screen.

Knowledge Sharing

- Presenter: Samar & Tejas
- Topic: "Async Background Orchestration: Bridging FastAPI, LangGraph `astream`, and SSE for Real-Time AI Telemetry."
- Duration: 35 minutes during Sunday online session.

Review

Nidhish reviews prompt response quality; Samar reviews async task lifecycle; Shreya audits database connection pool under load.

Documentation

Update `docs/09-apis/api-reference.md` with new task execution behavior and commit Week 7 sign-offs to `developer_plan.md`.

Expected Result

FastAPI fully powered by the compiled LangGraph state machine, streaming live execution steps to the frontend with database persistence.

Definition of Done

POST /api/chat launches asynchronous LangGraph execution; UI renders real-time step progression; PostgreSQL agent_steps records all node transitions.

WEEK 08

Secure Tool Sandbox, Self-RAG Critique & HITL Checkpoint

FRIDAY

Day 22

Friday — Day 22

Goal

Implement the isolated Secure Tool Sandbox (ADR-004) executing AI-generated Python numerical calculation scripts inside an ephemeral, network-denied Docker container.

Task Sequence

- DEV-022.1 [Shreya & Samar]: Build lightweight sandbox container image (`docker/sandbox.Dockerfile`) containing Python 3.11, `numpy`, `scipy`, and `openpyxl`.
- DEV-022.2 [Shreya]: Build Python AST pre-scanner inspecting generated code for dangerous builtins (`eval`, `exec`, `os.system`, `subprocess`, `socket`).
- DEV-022.3 [Samar]: Implement sandbox execution runner in `backend/app/tools/sandbox.py` using Docker SDK for Python with `--network none` and 512MB RAM cap.
- DEV-022.4 [Dakshit]: Implement standard industrial calculation scripts: Corrosion Rate (\$CR\$) and Remaining Service Life (\$RL\$) according to API 570.
- DEV-022.5 [Nidhish]: Craft code generation prompt instructing LLM to generate pure Python calculation functions without external network dependencies.
- DEV-022.6 [Tejas]: Build code execution card in frontend `MessageBubble.tsx` displaying the generated Python script, execution output, and sandbox status badge.
- DEV-022.7 [Safa]: Author test suite (`backend/tests/test_sandbox.py`) verifying execution of valid math scripts and termination of malicious/runaway scripts.

Developer Responsibilities (In Execution Order)

- Shreya (Sandbox Security Architect): Enforces `--network none`, 512MB memory ceilings, 10s timeouts, and writes the AST security scanner.
- Samar (Docker Runner Engineer): Implements container lifecycle management in `backend/app/tools/sandbox.py`, ensuring containers are destroyed immediately.
- Dakshit (Engineering Math Specialist): Writes the calculation formulas for corrosion rate (\$CR\$) and remaining life (\$RL\$).
- Nidhish (Code Prompt Engineer): Formulates code generation prompts enforcing standard Python math syntax without dangerous imports.
- Tejas (Code Execution UI Lead): Renders the generated script and execution results in a collapsible code block in the chat UI.
- Safa (Sandbox Security Tester): Executes attack scripts (fork bombs, socket attempts, filesystem probes) and verifies the sandbox blocks them.

Collaboration Flow

Shreya and Samar construct the hardened container runner; Dakshit and Nidhish provide engineering calculations and prompts; Tejas builds the UI display; Safa executes security boundary testing.

Prerequisites

- Week 7 completed.
- Docker daemon running locally.

Hands-On Requirement

Every developer must execute `python -m pytest backend/tests/test_sandbox.py`, verify that a valid corrosion calculation executes in $<1.5\text{ s}$, and verify that an attempted network socket script is immediately blocked.

Knowledge Sharing

- Presenter: Shreya & Samar
- Topic: "Hardened Sandboxes for Untrusted LLM Code: Docker Network Denial, cgroups Resource Quotas & AST Pre-Validation."
- Duration: 25 minutes during Friday offline sync.

Review

Shreya audits container isolation flags; Samar reviews container cleanup; Nidhish verifies math accuracy.

Documentation

Document sandbox architecture, security flags, and calculation formulas in `docs/05-system-design/tool-sandbox-design.md` and `developer_plan.md`.

Expected Result

A secure execution sandbox running numerical Python code with complete network denial and guaranteed resource termination.

Definition of Done

`test_sandbox.py` achieves 100% pass rate; malicious code is rejected by AST scanner; runaway infinite loops are killed at 10.0 seconds.

SATURDAY

Day 23

Saturday — Day 23

Goal

Implement the Self-RAG Critique Gate (ADR-006) algorithmically evaluating retrieval relevance (ISREL) and factual grounding (ISSUP), triggering automated revision loops upon failure.

Task Sequence

1. DEV-023.1 [Nidhish]: Implement critique evaluation node in `backend/app/agent/nodes/self_rag.py`.
2. DEV-023.2 [Nidhish]: Implement ISREL metric: evaluates whether retrieved SOP clauses are semantically relevant to the user query ($\text{Threshold} \geq 0.75$).
3. DEV-023.3 [Nidhish]: Implement ISSUP metric: evaluates whether every factual claim in the generated recommendation is supported by retrieved text ($\text{Threshold} \geq 0.80$).
4. DEV-023.4 [Samar]: Wire critique node into LangGraph state machine; manage conditional edge branching and revision counter limits (`max_retries=2`).
5. DEV-023.5 [Shreya]: Review hallucination detection thresholds; ensure safety-critical advice cannot bypass the critique gate.
6. DEV-023.6 [Safa]: Author test cases with intentional hallucinations (invented thickness numbers, wrong API clauses); verify critique gate catches 100% of them.
7. DEV-023.7 [Tejas]: Build visual verification badge in frontend showing ISREL and ISSUP scores, factual grounding status, and revision count.

Developer Responsibilities (In Execution Order)

1. Nidhish (Self-RAG Algorithm Lead): Implements the ISREL and ISSUP evaluation prompts and scoring functions in `self_rag.py`.
2. Samar (Cyclic Revision Edge Engineer): Wires the conditional branch in LangGraph, routing failed evaluations back to the planning node.
3. Shreya (Safety Threshold Reviewer): Audits threshold strictness, ensuring that ambiguous or ungrounded claims are never approved.
4. Dakshit (Inspection Grounding Specialist): Verifies that factual grounding checks accurately match API 570 Table 4 requirements.
5. Safa (Hallucination Test Specialist): Injects synthetic hallucinations into test drafts, validating that the critique gate intercepts 100% of them.
6. Tejas (Critique Badge UI Lead): Renders the ISREL/ISSUP verification scores and revision badges in the frontend chat bubble.

Collaboration Flow

Nidhish codes the critique metrics; Samar connects the revision loop; Shreya and Dakshit audit safety thresholds; Safa feeds synthetic hallucination tests; Tejas visualizes scores in the UI.

Prerequisites

- DEV-021 and DEV-022 completed.
- Understanding of ADR-006.

Hands-On Requirement

All 6 developers execute `pytest backend/tests/test_self_rag.py`, observe an ungrounded draft fail the ISSUP check, see the graph trigger revision, and watch the corrected draft pass verification.

Knowledge Sharing

- Presenter: Nidhish & Safa
- Topic: "Automated Hallucination Prevention: Implementing Self-RAG Critique Gates (ISREL & ISSUP) in Safety-Critical AI."
- Duration: 30 minutes during Saturday online session.

Review

Samar reviews graph loop termination; Nidhish reviews evaluation prompt consistency; Shreya audits threshold strictness.

Documentation

Document Self-RAG formulas, thresholds, and revision loop logic in `docs/07-ai-ml/self-rag-verification.md` and `developer_plan.md`.

Expected Result

An automated critique gate that detects ungrounded claims and silently rewrites recommendations before human presentation.

Definition of Done

Critique gate intercepts 100% of synthetic hallucination test cases; verified recommendations pass with ISREL ≥ 0.75 and ISSUP ≥ 0.80 .

SUNDAY

Day 24

Sunday — Day 24

Goal

Implement the deterministic Human-in-the-Loop (4-Eye) Approval Checkpoint, freezing the LangGraph state machine before hazardous recommendations and resuming upon operator sign-off.

Task Sequence

1. DEV-024.1 [Samar]: Configure LangGraph checkpointing with PostgreSQL checkpointer (AsyncPostgresSaver) in `backend/app/agent/graph.py`.
2. DEV-024.2 [Nidhish]: Implement risk assessment logic in `plan_node` flagging tasks as CRITICAL when remaining life $< 2.0 \times \text{years}$.
3. DEV-024.3 [Samar]: Configure graph interrupt: `interrupt_before=["human_checkpoint"]` when task risk level is evaluated as HIGH or CRITICAL.
4. DEV-024.4 [Tejas]: Polish frontend `ApprovalPanel.tsx` modal with diff viewer showing original vs edited recommendation, operator ID input, and sign-off buttons.
5. DEV-024.5 [Dakshit]: Verify that operator edits seamlessly overwrite the agent recommendation in state before document synthesis.
6. DEV-024.6 [Shreya]: Implement cryptographic signature hashing for approval events; store operator ID, timestamp, and decision hash in `approvals` table.
7. DEV-024.7 [Safa]: Execute end-to-end checkpoint test: pause execution, verify state survives backend restart, and resume successfully after 5 minutes.

Developer Responsibilities (In Execution Order)

1. Samar (Checkpoint State Architect): Configures `AsyncPostgresSaver`, implements the deterministic graph interrupt, and builds the resume handler.
2. Nidhish (Risk Classifier Specialist): Configures risk-scoring logic that flags hazardous corrosion rates and triggers mandatory human intervention.
3. Tejas (Approval Interface Lead): Builds the interactive `ApprovalPanel` modal, allowing operators to review, edit, or reject recommendations.
4. Dakshit (State Mutation Specialist): Ensures operator edits overwrite AI draft text cleanly before deliverable synthesis.
5. Shreya (Approval Cryptographer): Hashes operator decisions and IDs into cryptographic signature stamps in the PostgreSQL database.
6. Safa (Persistence & Recovery Tester): Tests state persistence by killing the server during an active pause, restarting, and verifying clean resumption.

Collaboration Flow

Samar and Nidhish implement the state pause and risk triggers; Tejas builds the modal; Dakshit verifies state updates; Shreya secures digital signatures; Safa tests crash resilience.

Prerequisites

- DEV-021, DEV-022, and DEV-023 completed.
- PostgreSQL running locally.

Hands-On Requirement

All 6 developers trigger a high-risk corrosion query in their UI, watch the execution pause at the checkpoint, inspect the frozen state in PostgreSQL, submit an approval, and watch the execution complete.

Knowledge Sharing

- Presenter: Samar & Shreya
- Topic: "Human-in-the-Loop Architecture: LangGraph Checkpoints, State Serialization & Cryptographic 4-Eye Approval Protocols."
- Duration: 35 minutes during Sunday online session.

Review

Tejas reviews approval modal UX; Samar reviews checkpointer serialization; Shreya verifies tamper-evident approval logging.

Documentation

Document checkpoint state lifecycle, interrupt configuration, and approval API schemas in `docs/04-architecture/architecture-overview.md` and `developer_plan.md`.

Expected Result

Deterministic pause/resume execution semantics allowing human operators to inspect, edit, or reject AI findings before deliverable generation.

Definition of Done

LangGraph halts execution at checkpoint; state persists safely in PostgreSQL; approval resumes execution and completes the workflow with 100% test pass rate.

WEEK 09

Deliverable Synthesis & Frontend Console Real-Time Integration

FRIDAY

Day 25

Friday — Day 25

Goal

Implement the publication-grade python-docx Deliverable Synthesizer, generating official corporate `Inspection_Approval_Note.docx` files complete with metadata tables, defect images, and SHA-256 stamps.

Task Sequence

- DEV-025.1 [Dakshit]: Build deliverable synthesizer in `backend/app/tools/docx_writer.py` utilizing official corporate styling guidelines.
- DEV-025.2 [Dakshit]: Implement document layout sections (refinery header, equipment table, NDT measurement table, embedded defect photo, API 570 citations, sign-off block).
- DEV-025.3 [Nidhish]: Format synthesized executive summary text and engineering recommendations for document inclusion.
- DEV-025.4 [Shreya]: Implement SHA-256 cryptographic stamping: ensure document hash is computed at creation and stored in database.
- DEV-025.5 [Samar]: Implement secure deliverable download endpoint `GET /api/tasks/{id}/download` streaming files with proper MIME types.
- DEV-025.6 [Tejas]: Build deliverable download card in frontend `MessageBubble.tsx` showing file name, size, SHA-256 hash stamp, and direct download button.
- DEV-025.7 [Safa]: Inspect generated Word documents in Microsoft Word / LibreOffice; verify layout compliance, margin spacing, and image aspect ratios.

Developer Responsibilities (In Execution Order)

- Dakshit (Document Generation Lead): Builds the Word document generator using python-docx, implementing corporate headers, tables, and image embedding.
- Nidhish (Executive Summary Formatter): Structures the AI recommendations and standards citations into publication-ready technical text.
- Shreya (Cryptographic Stamp Engineer): Calculates the SHA-256 checksum of the generated .docx file and binds it to the audit ledger.
- Samar (Download Stream Specialist): Implements `GET /api/tasks/{id}/download` in FastAPI with verified file headers.
- Tejas (Download Card UI Lead): Renders the document download card in the chat interface with file size and hash stamps.
- Safa (Document Quality Inspector): Tests generated files across MS Word and LibreOffice, verifying margins, table borders, and image resolution.

Collaboration Flow

Dakshit leads document styling; Nidhish formats the content; Shreya adds cryptographic hashes; Samar builds the download route; Tejas renders the UI card; Safa performs visual QA.

Prerequisites

- Week 8 completed.
- Python python-docx installed.

Hands-On Requirement

All 6 developers execute `python -m pytest backend/tests/test_docx_writer.py`, download the generated .docx file, open it in Word/LibreOffice, and verify formatting and tables.

Knowledge Sharing

- Presenter: Dakshit & Safa
- Topic: "Automating Executive Engineering Deliverables: python-docx Templating, Embedded Visual Evidence & Digital Signature Stamping."
- Duration: 25 minutes during Friday offline sync.

Review

Safa reviews document visual aesthetics; Samar reviews streaming endpoints; Shreya audits cryptographic hash matching.

Documentation

Document report layout templates, table specifications, and download API schemas in `docs/05-system-design/` and `developer_plan.md`.

Expected Result

Automated synthesis of official, publication-quality Word inspection reports ready for statutory presentation and executive sign-off.

Definition of Done

Generated .docx opens without errors in MS Word; contains all tables, images, and citations; matches SHA-256 hash in database with 100% test pass rate.

SATURDAY

Day 26

Saturday — Day 26

Goal

Perform complete frontend consolidation, integrating the Next.js 16 3-column AppShell with live SSE event streams, Model Router cards, RAG citation cards, and interactive approval panels.

Task Sequence

1. DEV-026.1 [Tejas]: Connect central Zustand store (`useTaskStore.ts`) to live SSE stream, eliminating all remaining mock data dependencies.
2. DEV-026.2 [Tejas]: Polish the 3-column industrial layout (Sidebar templates, Center chat workspace, Right ContextPanel).
3. DEV-026.3 [Dakshit]: Integrate interactive Recharts graphics: render wall-thickness degradation curves and vibration FFT spectrum charts inline.
4. DEV-026.4 [Nidhish]: Review chat feed typography and markdown rendering; ensure code blocks, tables, and mathematical formulas display with high contrast.
5. DEV-026.5 [Samar]: Verify that backend SSE stream feeds all necessary properties to the frontend store without requiring secondary REST polling calls.
6. DEV-026.6 [Shreya]: Audit frontend security: check for XSS vulnerabilities in markdown renderer, sanitize raw HTML inputs, verify strict Content-Security-Policy.

7. DEV-026.7 [Safa]: Conduct usability audit across multiple screen resolutions (1920x1080, 1440x900, 1366x768); ensure 3-column layout collapses gracefully.

Developer Responsibilities (In Execution Order)

1. Tejas (Frontend Consolidation Lead): Binds the Zustand store to live backend SSE events and polishes the Black + Safety Orange theme (#FF6A00).
2. Dakshit (Data Visualization Specialist): Builds dynamic Recharts components rendering degradation curves and threshold overlay lines.
3. Nidhish (Typography & Content Auditor): Validates markdown rendering for engineering formulas, code snippets, and table layouts.
4. Samar (API Optimization Specialist): Ensures SSE event payloads provide complete state snapshots, eliminating redundant client REST calls.
5. Shreya (Client Security Auditor): Audits DOM rendering for XSS vulnerabilities, sanitizing user inputs and enforcing strict CSP headers.
6. Safa (Responsive Design Tester): Tests the UI across various display resolutions and validates mobile/tablet responsive behavior.

Collaboration Flow

Tejas leads UI consolidation; Dakshit integrates charts; Nidhish audits typography; Samar optimizes payloads; Shreya enforces client security; Safa validates responsiveness.

Prerequisites

- DEV-024 and DEV-025 completed.
- Next.js 16 frontend running locally.

Hands-On Requirement

All 6 developers execute `npm run build` in frontend/, verify 0 errors, open the application in their browser, and test the responsive sidebar and context panel toggles.

Knowledge Sharing

- Presenter: Tejas
- Topic: "Industrial UI Design Systems: High-Density Layouts, Safety Orange Telemetry (#FF6A00), Monospace Data Displays & Zero Mock Dependencies."
- Duration: 30 minutes during Saturday online session.

Review

Tejas and Safa review visual design consistency; Shreya reviews DOM sanitization; Samar verifies WebSocket/SSE connection efficiency.

Documentation

Document UI component hierarchy, color tokens, and state store architecture in `docs/04-architecture/architecture-overview.md` and `developer_plan.md`.

Expected Result

A production-grade, responsive Next.js 16 industrial workbench UI completely driven by live backend events with zero mock dependencies.

Definition of Done

`npm run build` compiles with 0 TypeScript/ESLint warnings; all interactive components respond to live backend events; zero mock files referenced in production paths.

SUNDAY

Day 27

Sunday — Day 27

Goal

Execute the Flagship Golden Path (NDT Corrosion Audit) end-to-end across all 6 developer machines, validating the full architectural chain from upload to document download.

Task Sequence

1. DEV-027.1 [Tejas]: Execute intake: upload scanned NDT report `HC_102_B_UT_Inspection_Report.pdf` and defect photo `corrosion_flange.png` in UI.
2. DEV-027.2 [Dakshit]: Verify PaddleOCR extracts wall-thickness table (Point P-01: 3.20 mm vs 6.02 mm nominal).
3. DEV-027.3 [Nidhish]: Verify Adaptive Model Router selects Qwen2.5-VL and Qwen2.5-14B with explainable rationale displayed in UI.
4. DEV-027.4 [Dakshit]: Verify Hybrid RAG retrieves API 570 Section 7 Table 4 from Qdrant with citation card rendered.
5. DEV-027.5 [Samar]: Verify Docker sandbox executes corrosion rate calculation ($CR = 0.564\text{ mm/yr}$, $RL = 1.24\text{ yrs}$) with `--network none`.
6. DEV-027.6 [Nidhish]: Verify Self-RAG critique passes (ISREL = 0.95, ISSUP = 1.00).
7. DEV-027.7 [Tejas]: Verify execution pauses at 4-Eye Approval Checkpoint; operator edits replacement window to 6 months and approves.
8. DEV-027.8 [Dakshit]: Verify synthesized Word note downloads cleanly with embedded charts, tables, and SHA-256 stamp.
9. DEV-027.9 [Shreya]: Monitor socket state during entire workflow; confirm 0 outbound network bytes sent; verify cryptographic audit chain integrity.
10. DEV-027.10 [Safa]: Record stopwatch execution time for each step; verify total workflow completion time is $<60\text{ seconds}$.

Developer Responsibilities (In Execution Order)

1. Tejas (Golden Path Operator): Drives the UI workflow, uploads inspection documents, and validates node illumination on the timeline.
2. Dakshit (OCR & RAG Validator): Validates tabular extraction numbers against the original physical scan and verifies SOP citation accuracy.
3. Nidhish (AI Reasoning Validator): Validates router model assignment, prompt context formatting, and critique score thresholds.
4. Samar (Backend & Sandbox Monitor): Traces backend server logs, container execution in the sandbox, and state persistence.
5. Shreya (Zero-Egress & Audit Verifier): Verifies socket monitors report 0 outbound bytes and validates forward hash chaining in the audit database.
6. Safa (Workflow Benchmark Scribe): Logs execution durations for every stage, ensuring total latency remains under 60 seconds.

Collaboration Flow

All 6 developers execute the complete 9-step workflow concurrently on their own laptops, validating data accuracy, streaming responsiveness, and audit persistence.

Prerequisites

- All Week 1 through Week 9 subsystems operational.
- Sample inspection files loaded.

Hands-On Requirement

All 6 developers must personally execute the full 9-step flagship workflow on their laptops, download the generated .docx file, and verify the task reaches "Completed" status.

Knowledge Sharing

- Presenter: All 6 Developers (Round-Robin)
- Topic: "The Flagship Golden Path: Demonstrating the Complete End-to-End Sovereign Agentic Lifecycle from Scanned PDF to Verified Sign-Off."
- Duration: 35 minutes during Sunday online session.

Review

Each developer reviews the generated deliverable on a peer's machine; Shreya runs the audit verifier across all 6 databases.

Documentation

Record step-by-step flagship execution timestamps and screenshots in docs/13-sih/flagship-demo-script.md and commit Week 9 milestone to developer_plan.md.

Expected Result

Flawless end-to-end execution of the primary industrial inspection workflow on all 6 laptops with full audit verification.

Definition of Done

100% of team members successfully complete the flagship workflow on their local machines with verified .docx deliverable and zero errors.

WEEK 10

Zero-Egress Network Sentinel & Master Containerization

FRIDAY

Day 28

Friday — Day 28

Goal

Implement the Zero-Egress Network Sentinel monitoring module in the backend, continuously inspecting kernel sockets via `/proc/net/tcp` and `psutil` to guarantee zero outbound network traffic.

Task Sequence

1. DEV-028.1 [Shreya]: Implement kernel socket inspection worker in `backend/app/network_sentinel/monitor.py`.
2. DEV-028.2 [Shreya]: Implement `/proc/net/tcp` and `psutil.net_connections()` poller scanning all established, `syn_sent`, and listening sockets.
3. DEV-028.3 [Samar]: Implement background async polling loop in FastAPI lifespan event, running socket audits every 2.0 seconds with minimal CPU footprint ($<1\%$).
4. DEV-028.4 [Nidhish & Dakshit]: Test Ollama and Qdrant loopback sockets; ensure local inference sockets are recognized as authorized internal traffic.
5. DEV-028.5 [Shreya]: Define whitelist of permissible internal socket bindings (localhost `127.0.0.1`, container bridge network `172.x.x.x`).
6. DEV-028.6 [Safa]: Author simulated breach script (`scripts/simulate_leak_attempt.py`) attempting an external connection to `8.8.8.8:53` and verify immediate detection.
7. DEV-028.7 [Tejas]: Connect `NetworkSentinelView.tsx` widget in UI to live `/api/network/status` endpoint, displaying green "AIR-GAP LOCKED" status and live packet counters.

Developer Responsibilities (In Execution Order)

1. Shreya (Kernel Socket Sentinel Architect): Implements socket inspection algorithms reading `/proc/net/tcp` and `psutil`, establishing internal IP whitelists.
2. Samar (Background Poller Engineer): Integrates the socket poller into FastAPI's background lifespan runner, maintaining low CPU usage.
3. Nidhish (LLM Loopback Auditor): Verifies that local Ollama socket traffic on port 11434 is cleanly whitelisted.
4. Dakshit (Vector Store Loopback Auditor): Verifies that local Qdrant socket traffic on port 6333 is cleanly whitelisted.
5. Safa (Breach Simulation Specialist): Executes simulated external connection attempts, confirming Sentinel intercepts and logs the unauthorized socket.
6. Tejas (Sentinel UI Lead): Binds the live network telemetry data to the `NetworkSentinelView` widget in the frontend.

Collaboration Flow

Shreya and Samar build the socket monitor; Nidhish and Dakshit test service whitelisting; Safa executes breach tests; Tejas visualizes telemetry in the UI.

Prerequisites

- Week 9 completed.
- Python `psutil` installed.

Hands-On Requirement

Every developer runs `python scripts/test_network_sentinel.py`, runs the simulated breach attempt, and verifies that Sentinel catches the foreign socket within 2.0 seconds and records it in `audit_logs`.

Knowledge Sharing

- Presenter: Shreya & Samar
- Topic: "Kernel-Level Zero-Egress Auditing: Reading `/proc/net/tcp`, Socket Whitelisting & Proving Air-Gap Sovereignty."
- Duration: 25 minutes during Friday offline sync.

Review

Samar reviews poller CPU utilization; Shreya verifies foreign socket interception; Tejas checks UI telemetry refresh rate.

Documentation

Document Sentinel architecture, socket whitelists, and telemetry APIs in `docs/04-architecture/zero-egress-enclave.md` and `developer_plan.md`.

Expected Result

An active kernel socket monitor continuously proving zero external network egress and exposing live telemetry to the UI.

Definition of Done

Sentinel successfully detects unauthorized external socket attempts, logs the event, and maintains 0 outbound bytes verified across all 6 machines.

SATURDAY
Y

Day 29

Saturday — Day 29

Goal

Implement host-level firewall enforcement scripts (`iptables/ufw`) establishing a default-deny outbound network policy, and validate system execution under physical network disconnection.

Task Sequence

1. DEV-029.1 [Shreya]: Author host firewall configuration script `scripts/enforce_airgap.sh` establishing default-deny outbound policy.
2. DEV-029.2 [Shreya]: Author firewall teardown/recovery script `scripts/restore_network.sh` for development flexibility.
3. DEV-029.3 [Samar]: Test database, FastAPI, and SSE operation under strict `iptables` drop policy; verify internal communication functions unimpeded.
4. DEV-029.4 [Tejas]: Test frontend operation while machine is completely offline; verify fonts, CSS, and icons load from local bundles with zero CDN dependencies.
5. DEV-029.5 [Dakshit]: Test document parsing and OCR while network is disabled; confirm no dynamic model downloads are triggered.
6. DEV-029.6 [Nidhish]: Verify Ollama and local LLMs execute offline without attempting HuggingFace or license server check-ins.
7. DEV-029.7 [Safa]: Execute the physical cable-pull test; document step-by-step verification evidence and packet capture logs in the security report.

Developer Responsibilities (In Execution Order)

1. Shreya (Firewall Engineer & Packet Analyst): Authors `scripts/enforce_airgap.sh` with default-deny `iptables` rules and captures traffic using `tcpdump`.

2. Samar (Internal Network Verifier): Verifies that container bridge networking and localhost HTTP/SSE calls operate seamlessly under firewall drop rules.
3. Tejas (Offline Asset Auditor): Verifies that all UI fonts, Tailwind styles, and icons render locally with zero external CDN requests.
4. Dakshit (Offline Vision Pipeline Auditor): Verifies that PaddleOCR and document extractors run with zero external network access.
5. Nidhish (Offline LLM Inference Auditor): Verifies that Ollama runs inferences completely offline without external license check-ins.
6. Safa (Physical Air-Gap Test Specialist): Unplugs physical Ethernet cables, disables Wi-Fi, runs the full inspection flow, and logs the results.

Collaboration Flow

Shreya writes and applies the firewall rules; Samar, Tejas, Dakshit, and Nidhish verify subsystem independence; Safa conducts physical disconnect tests.

Prerequisites

- DEV-028 completed.
- Root/administrator access to manage firewall rules.

Hands-On Requirement

All 6 developers must run `sudo bash scripts/enforce_airgap.sh` (or Windows firewall equivalent), unplug Ethernet, turn off Wi-Fi, run the flagship NDT workflow in their browser, and verify 100% success.

Knowledge Sharing

- Presenter: Shreya
- Topic: "Hard Air-Gap Defense: iptables Default-Deny Policies, Packet Capture Proof & Surviving Physical Disconnection."
- Duration: 30 minutes during Saturday online session.

Review

Shreya reviews `tcpdump` packet capture logs; Samar verifies localhost routing; Tejas confirms local asset loading.

Documentation

Document firewall scripts, packet capture evidence, and air-gap verification protocols in `docs/11-testing/zero-egress-validation.md` and `developer_plan.md`.

Expected Result

Definitive, reproducible proof that ABHEDYA AI runs with zero external packets and operates flawlessly with physical networking disconnected.

Definition of Done

`verify_zero_egress.sh` confirms 0 outbound external packets during complete flagship workflow execution on all 6 laptops.

SUNDAY

Day 30

Sunday — Day 30

Goal

Architect and validate the unified `Master docker-compose.yml` orchestrating all 5 microservices (frontend, backend, db, qdrant, ollama) over a local isolated bridge network with single-command cold boot.

Task Sequence

1. DEV-030.1 [Samar & Shreya]: Create unified root `docker-compose.yml` defining all 5 services, volume mounts, health checks, and restart policies.
2. DEV-030.2 [Shreya]: Configure internal non-routable bridge network `abhedya_net` with external gateway access disabled (`internal: true`).
3. DEV-030.3 [Samar]: Optimize backend/Dockerfile with non-root user execution, multi-stage builds, and health check `curl` command.
4. DEV-030.4 [Tejas]: Optimize frontend/Dockerfile for multi-stage standalone Next.js production build (`node:20-alpine`), minimizing image size to $<180\text{MB}$.
5. DEV-030.5 [Dakshit]: Verify Qdrant volume persistence across container restarts; ensure indexed SOP vectors survive container teardown.
6. DEV-030.6 [Nidhish]: Verify Ollama container volume mounts; ensure pre-pulled model weights in `ollama_models` are recognized immediately without re-downloading.
7. DEV-030.7 [Safa]: Execute cold-boot benchmark across all 6 laptops; record startup duration; document container troubleshooting steps.

Developer Responsibilities (In Execution Order)

1. Samar (Compose Infrastructure Lead): Assembles the master `docker-compose.yml`, configuring service dependencies, health checks, and restart policies.
2. Shreya (Container Network Security Lead): Configures `abhedya_net` with `internal: true`, verifying that containers cannot route packets to the host's internet interface.
3. Tejas (Frontend Containerization Lead): Builds the multi-stage Alpine Dockerfile for Next.js, producing a lightweight, standalone image.
4. Dakshit (Vector Storage Persistence Lead): Verifies that Qdrant indexed points persist across `docker compose down` and `up` cycles.
5. Nidhish (Model Storage Persistence Lead): Configures Docker named volumes for Ollama, ensuring model weights remain cached across boots.
6. Safa (Cold-Boot Benchmark Specialist): Times cold-boot startup durations on all 6 laptops and compiles the troubleshooting runbook.

Collaboration Flow

Samar and Shreya architect the compose stack; Tejas packages the frontend; Dakshit and Nidhish configure persistent volumes; Safa tests cold boots.

Prerequisites

- DEV-028 and DEV-029 completed.
- Docker Compose v2 installed.

Hands-On Requirement

All 6 developers must run:

```
docker compose down -v
docker compose up -d
docker compose ps
```

and verify all 5 containers reach `healthy` status and the application is accessible at `http://localhost:3000`.

Knowledge Sharing

- Presenter: Samar & Shreya
- Topic: "Production Container Orchestration: Multi-Stage Dockerfiles, Health Checks, Dependency Gating & Isolated Bridge Networks."
- Duration: 35 minutes during Sunday online session.

Review

Tejas reviews frontend container build; Samar reviews backend health checks; Shreya audits container network isolation.

Documentation

Document Docker Compose architecture, service ports, volume mappings, and deployment commands in `docs/12-deployment/compose-deployment.md` and `developer_plan.md`.

Expected Result

A turnkey, production-grade Docker Compose stack launching the entire ABHEDYA AI platform with a single command in $<60\text{ seconds}$.

Definition of Done

`docker compose up -d` brings all 5 containers to healthy state; full application functions flawlessly across all 6 developer machines.

WEEK 11

Security Hardening, Resilience Testing & SIH Rehearsal

FRIDAY

Day 31

Friday — Day 31

Goal

Execute comprehensive automated regression testing, security vulnerability scanning, and negative test case evaluation across all platform layers.

Task Sequence

1. DEV-031.1 [Samar]: Consolidate all unit, integration, and security tests into a single master test runner `scripts/run_all_tests.sh`.
2. DEV-031.2 [Shreya]: Execute static application security testing (SAST) using `bandit` (Python backend) and `npm audit` (Next.js frontend).
3. DEV-031.3 [Dakshit]: Test OCR pipeline with corrupted and blank images; verify system returns structured error rather than unhandled exception.
4. DEV-031.4 [Nidhish]: Test model router with ambiguous, multilingual, and nonsensical queries; verify fallback routing behaves deterministically.
5. DEV-031.5 [Tejas]: Run frontend test suite (`npm run test`); verify error boundary components catch unexpected rendering errors gracefully.
6. DEV-031.6 [Safa]: Execute negative test matrix; log every error response code; verify all security rejections write to PostgreSQL `audit_logs`.

Developer Responsibilities (In Execution Order)

1. Samar (Master Test Automation Lead): Builds `scripts/run_all_tests.sh`, consolidating backend unit tests, integration tests, and API checks.
2. Shreya (Static Security Analysis Lead): Runs SAST scanners (`bandit`, `npm audit`), resolving dependency vulnerabilities and checking for unsafe patterns.
3. Dakshit (Input Fuzzing Specialist): Submits corrupted, truncated, and blank PDF/image files to test upload and OCR resilience.
4. Nidhish (Adversarial Prompt Fuzzer): Submits ambiguous and adversarial queries to verify the router and policy engine fail safely.
5. Tejas (Frontend Error Boundary Lead): Tests React error boundaries, ensuring UI displays clean error cards rather than blank screens.
6. Safa (Regression Matrix Auditor): Executes the full regression suite across all machines and compiles the QA compliance report.

Collaboration Flow

Samar organizes the master test runner; Shreya conducts security scans; Dakshit, Nidhish, and Tejas test subsystem edge cases; Safa audits results.

Prerequisites

- Week 10 completed.

- Full Docker stack running locally.

Hands-On Requirement

All 6 developers execute `bash scripts/run_all_tests.sh` on their local workstations and confirm that all 65+ tests pass with zero failures.

Knowledge Sharing

- Presenter: Samar & Safa
- Topic: "Automated Regression Testing & Negative Test Engineering: Ensuring Industrial Resilience Against Malformed Data and Adversarial Inputs."
- Duration: 25 minutes during Friday offline sync.

Review

Shreya reviews SAST vulnerability reports; Samar reviews test coverage reports; Tejas reviews error boundaries.

Documentation

Publish complete test execution report and security audit findings in `docs/11-testing/testing-strategy.md` and `developer_plan.md`.

Expected Result

A hardened, audited codebase passing 100% of automated tests with zero security vulnerabilities and verified negative error handling.

Definition of Done

`bash scripts/run_all_tests.sh` passes 100% on all 6 laptops with 0 failures, 0 skipped critical tests, and 0 high-severity security warnings.

SATURDAY

Day 32

Saturday — Day 32

Goal

Perform fault-injection stress testing, simulating container crashes, database disconnects, and VRAM exhaustion, and validate self-healing recovery mechanisms.

Task Sequence

1. DEV-032.1 [Samar]: Execute simulated database crash: kill PostgreSQL container mid-transaction, restart container, and verify database auto-recovers with zero corrupted records.
2. DEV-032.2 [Nidhish]: Execute simulated Ollama crash: terminate Ollama process during active inference, verify backend returns clean timeout event, and test auto-reconnection.
3. DEV-032.3 [Nidhish]: Implement request queue throttling in `llm.py` preventing concurrent inferences from triggering CUDA OOM crashes.
4. DEV-032.4 [Tejas]: Test frontend reconnection resiliency: verify UI reconnects to SSE stream upon server restart without duplicate message rendering.
5. DEV-032.5 [Dakshit]: Test document re-processing resilience: ensure re-uploading the same file handles deduplication cleanly.
6. DEV-032.6 [Shreya]: Validate PostgreSQL backup script (`pg_dump` with encryption); test full database restoration onto a clean instance.
7. DEV-032.7 [Safa]: Document recovery time objectives (RTO) and recovery point objectives (RPO) for each simulated failure scenario.

Developer Responsibilities (In Execution Order)

1. Samar (Database Crash & Recovery Lead): Simulates database failures, testing connection pool re-establishment and transaction recovery.
2. Nidhish (LLM Crash & Throttling Lead): Simulates model inference crashes, implementing request throttling to prevent CUDA memory panics.
3. Tejas (Client Reconnect Specialist): Implements exponential backoff on SSE client reconnects, maintaining conversation history.
4. Dakshit (Data Deduplication Specialist): Tests file re-ingestion, verifying hash checks prevent duplicate processing.
5. Shreya (Disaster Recovery Lead): Authors and tests `scripts/backup_db.sh`, verifying encrypted database dumps can be restored in <30 seconds.
6. Safa (RTO/RPO Metrics Scribe): Measures recovery times for each injected failure scenario and compiles the resilience report.

Collaboration Flow

Samar and Nidhish inject backend crashes; Tejas and Dakshit ensure client and data stability; Shreya executes database restores; Safa logs RTO metrics.

Prerequisites

- DEV-031 completed.
- Master container stack operational.

Hands-On Requirement

Every developer runs the fault injection script `scripts/simulate_faults.sh`, observes the system handle container termination, and verifies automated recovery within 15 seconds.

Knowledge Sharing

- Presenter: Shreya & Samar
- Topic: "Chaos Engineering in Industrial AI: Fault Injection, Connection Pool Resilience & Instant Disaster Recovery."
- Duration: 30 minutes during Saturday online session.

Review

Samar reviews database recovery integrity; Shreya verifies encrypted backup restoration; Tejas checks frontend reconnect states.

Documentation

Document fault injection test results, recovery procedures, and backup instructions in `docs/13-sih/backup-contingency-plan.md` and `developer_plan.md`.

Expected Result

A fault-tolerant sovereign architecture capable of surviving service crashes and recovering state automatically within seconds.

Definition of Done

System recovers from simulated database and model crashes in <15 seconds with zero data loss and clean state restoration verified.

SUNDAY

Day 33

Sunday — Day 33

Goal

Implement the 1-Click Demo Mode, pre-warm local models, and execute 5 consecutive flawless dry-run rehearsals of the flagship SIH presentation under strict competition time constraints.

Task Sequence

1. DEV-033.1 [Samar]: Create automated 1-Click Demo launcher script `scripts/start_demo.sh` that boots containers, pre-warms Ollama models, and opens the browser.
2. DEV-033.2 [Nidhish]: Implement model pre-warming: send lightweight warm-up ping to load Qwen-14B and Qwen-VL weights into GPU VRAM before presentation.
3. DEV-033.3 [Tejas]: Lead the UI presentation walkthrough; rehearse screen narration and interactive feature highlighting; pace operator clicks.
4. DEV-033.4 [Dakshit]: Present the multimodal extraction and OCR table accuracy segment; explain engineering value over manual review.
5. DEV-033.5 [Shreya]: Present the Zero-Egress Network Sentinel, host firewall proof, and PostgreSQL forward-chained audit ledger segment.
6. DEV-033.6 [Safa]: Timekeeper and presentation auditor: log exact seconds per segment, identify awkward transitions, and refine speaking script.

Developer Responsibilities (In Execution Order)

1. Samar (Demo Automation Engineer): Authors `scripts/start_demo.sh` ensuring all services boot and reach readiness with one command.
2. Nidhish (Model Pre-Warmer & Speaker): Wires model pre-warming to eliminate first-token inference latency, and presents the Adaptive Router segment.
3. Tejas (Lead UI Presenter): Drives the live browser demonstration, pacing navigation and highlighting key industrial UI elements.
4. Dakshit (Multimodal Presenter): Demonstrates PaddleOCR extraction and Qwen-VL defect identification to technical judges.
5. Shreya (Security & Sovereignty Presenter): Demonstrates Sentinel 0-byte packet counters, firewall drop rules, and cryptographic audit chaining.
6. Safa (Rehearsal Director & Timekeeper): Enforces the 3-minute 30-second time limit, critiques presentation pacing, and tracks dry-run scores.

Collaboration Flow

The entire team executes 5 consecutive full rehearsals in rotation, practicing seamless verbal handoffs and verifying zero-latency execution.

Prerequisites

- All previous development tasks completed.
- Full stack operating on all laptops.

Hands-On Requirement

All 6 developers must execute `scripts/start_demo.sh`, launch the demo environment in under 30 seconds, and deliver the presentation script within the 3:30 time budget.

Knowledge Sharing

- Presenter: All 6 Developers
- Topic: "SIH Presentation Mastery: Pitching Sovereign AI to Technical Judges, Seamless Live Demos & Eliminating Stage Latency."
- Duration: 35 minutes during Sunday online session.

Review

Team reviews video recordings of rehearsals; optimizes narration cadence; verifies all contingency assets exist.

Documentation

Publish finalized spoken script, slide timings, and contingency triggers in `docs/13-sih/flagship-demo-script.md` and `developer_plan.md`.

Expected Result

A polished, synchronized 3.5-minute flagship live demonstration running flawlessly from a single command across all 6 laptops.

Definition of Done

Team completes 5 consecutive flawless dry runs within the 3:30 time limit with zero technical hitches or unhandled exceptions.

WEEK 12

Individual Verification, Technical Defense & Handover

FRIDAY

Day 34

Friday — Day 34

Goal

Freeze all production code, finalize user and administrator documentation, create offline air-gap installation packages, and record a 4K backup demonstration video.

Task Sequence

1. DEV-034.1 [Samar]: Enforce strict production code freeze on main branch; tag release v1.0.0-Abhedya-Production.
2. DEV-034.2 [Samar & Shreya]: Package complete offline air-gapped deployment bundle `scripts/package_airgap_bundle.sh` (Docker images, Ollama models, Qdrant vectors, SQL dump).
3. DEV-034.3 [Shreya]: Record a high-resolution 4K backup demonstration video with clear audio commentary covering the complete flagship workflow.
4. DEV-034.4 [Dakshit & Nidhish]: Finalize multimodal, RAG, and model registry documentation across `docs/06-data/` and `docs/07-ai-ml/`.
5. DEV-034.5 [Tejas]: Review all frontend documentation; ensure screenshots and UI architecture diagrams match the final production application.
6. DEV-034.6 [Safa]: Conduct documentation audit: verify every link, code block, and API signature in `docs/` is accurate and functional.

Developer Responsibilities (In Execution Order)

1. Samar (Release Manager): Enforces branch freeze, cuts release tag v1.0.0-Abhedya-Production, and packages container tarballs.
2. Shreya (Air-Gap Packager & Video Producer): Creates the self-contained offline deployment bundle and records the 4K backup demonstration video.
3. Dakshit (AI/ML Documentation Lead): Finalizes technical docs for PaddleOCR, Qwen-VL, Qdrant vector retrieval, and SOP corpora.
4. Nidhish (Router & Model Documentation Lead): Finalizes documentation for the Adaptive Model Router and Constitutional Policy guardrails.
5. Tejas (Frontend Documentation Lead): Updates screenshots, color token specs, and UI component hierarchy in the documentation hub.
6. Safa (Master Documentation Auditor): Conducts automated link-checking and markdown formatting audits across all 16 documentation directories.

Collaboration Flow

Samar and Shreya package the offline distribution assets and produce the video; Dakshit, Nidhish, and Tejas finalize technical docs; Safa audits all documentation links.

Prerequisites

- Week 11 completed.

- Codebase passing 100% of tests.

Hands-On Requirement

Every developer pulls the v1.0.0-Abhedya-Production tag, verifies the offline package script builds successfully, and tests loading the pre-recorded video.

Knowledge Sharing

- Presenter: Shreya & Samar
- Topic: "Air-Gapped Software Distribution: Packaging Self-Contained Enclave Tarballs for Zero-Internet Industrial Facilities."
- Duration: 25 minutes during Friday offline sync.

Review

All 6 developers sign off on the production code freeze; Safa confirms zero broken links across all markdown documentation.

Documentation

Update docs/changelog.md with the official v1.0.0 release notes and publish offline installation manual in docs/12-deployment/airgap-packaging.md.

Expected Result

A frozen, tagged production release packaged as an offline self-contained bundle with 4K backup demonstration video.

Definition of Done

v1.0.0-Abhedya-Production tagged; offline bundle verified with SHA-256 checksum; documentation audit passes 100%.

SATURDAY

Day 35

Saturday — Day 35

Goal

Conduct the Independent Fresh-Clone Verification Challenge: every developer wipes their local development environment and proves they can independently set up, run, and modify the system from scratch.

Task Sequence

1. DEV-035.1 [All 6 Developers]: Environment wipe: every developer deletes their local venv/, node_modules/, and local database volumes.
2. DEV-035.2 [All 6 Developers]: Fresh clone challenge: each developer performs a clean clone of the repository into a new directory (abhedya_fresh/).
3. DEV-035.3 [All 6 Developers]: Independent configuration: each developer builds virtual environments, installs dependencies, applies database migrations, and boots the platform without assistance.
4. DEV-035.4 [Cross-Assigned Injected Bug Challenges]:
 - Tejas: Diagnoses and fixes a broken SQL migration script in backend.
 - Dakshit: Diagnoses and fixes an SSE connection drop in frontend.
 - Nidhish: Diagnoses and fixes an iptables syntax error in security scripts.
 - Samar: Diagnoses and fixes a prompt injection vulnerability in policy engine.
 - Shreya: Diagnoses and fixes an OCR bounding-box parsing error.
 - Safa: Diagnoses and fixes a Docker sandbox memory-limit exception.
5. DEV-035.5 [All 6 Developers]: Every developer fixes their injected bug, verifies tests pass, and demonstrates the working system to the team.

Developer Responsibilities (In Execution Order)

1. Tejas (Cross-Domain Challenge: Database Migration): Performs fresh clone setup; diagnoses and fixes an injected SQL migration bug; explains fix to Samar.
2. Dakshit (Cross-Domain Challenge: Frontend SSE Streaming): Performs fresh clone setup; diagnoses and fixes an injected SSE connection drop; explains fix to Tejas.
3. Nidhish (Cross-Domain Challenge: Firewall Security): Performs fresh clone setup; diagnoses and fixes an injected iptables rule error; explains fix to Shreya.
4. Samar (Cross-Domain Challenge: Constitutional Policy): Performs fresh clone setup; diagnoses and fixes an injected prompt injection vulnerability; explains fix to Nidhish.
5. Shreya (Cross-Domain Challenge: OCR Table Parsing): Performs fresh clone setup; diagnoses and fixes an injected bounding box calculation error; explains fix to Dakshit.
6. Safa (Cross-Domain Challenge: Docker Sandbox Execution): Performs fresh clone setup; diagnoses and fixes an injected container cgroups memory exception; explains fix to Samar.

Collaboration Flow

Each developer works completely solo for 60 minutes to complete the fresh clone and solve their debugging challenge, then demonstrates their fix to the peer who originally built that component.

Prerequisites

- DEV-034 completed.
- Fresh working directory created.

Hands-On Requirement

All 6 developers must complete the fresh clone setup, fix their assigned synthetic bug, run the full test suite with 100% pass rate, and demonstrate the working UI on their machine.

Knowledge Sharing

- Presenter: All 6 Developers
- Topic: "Cross-Domain Competence: Proving Every Developer Can Independently Deploy, Troubleshoot, and Modify Any Layer of the System."
- Duration: 45 minutes during Saturday online session.

Review

Each developer's fix is peer-reviewed by the original author of that subsystem to confirm high technical accuracy.

Documentation

Record the results of the fresh-clone verification and challenge solutions in `docs/15-team-knowledge/rotation-matrix.md` and `developer_plan.md`.

Expected Result

Unquestionable proof that all 6 developers possess comprehensive end-to-end knowledge and can operate, debug, and maintain every subsystem independently.

Definition of Done

All 6 developers successfully boot the application from a fresh clone and resolve their cross-domain debugging challenge with 100% test pass rate.

SUNDAY

Day 36

Sunday — Day 36

Goal

Conduct the Mock SIH Grand Finale Jury Defense Simulation, subjecting every developer to rigorous cross-examination across all 6 technical mastery levels, followed by final project sign-off.

Task Sequence

- DEV-036.1 [All 6 Developers]: Set up formal jury defense simulation with timed technical cross-examination across the 6 Mastery Levels:
 - Level 1 (Product & Industrial Why): Why cloud AI is banned in refineries, OISD/API statutory compliance, and refinery safety economics.
 - Level 2 (Architectural Blueprint): Drawing all 9 layers from memory; data lifecycle of an NDT log; trust perimeters.
 - Level 3 (Code & Implementation): Explaining LangGraph StateGraph, FastAPI async workers, Qdrant vector retrieval, and Next.js SSE integration.
 - Level 4 (Security & Air-Gap Proof): Defending iptables rules, /proc/net/tcp socket polling, AST code sandboxing, and SHA-256 audit chaining.
 - Level 5 (Operations & Cold-Boot Recovery): Live cold boot from terminal; diagnosing simulated runtime failures in `<60\text{ seconds}`\$.
 - Level 6 (Personal Contribution): Articulating specific code contributions, architectural trade-offs, and systemic alignment.
- DEV-036.2 [All 6 Developers]: Conduct rapid-fire defense grilling: each developer defends questions outside their primary track.
- DEV-036.3 [All 6 Developers]: Complete the final comprehensive system sign-off and complete the 18-point Project Completion Checklist.
- DEV-036.4 [Samar & Safa]: Archive all development logs, update repository documentation, and commit final `developer_plan.md`.
- DEV-036.5 [Tejas]: Formal handover to team lead and final preparation for competition day.

Developer Responsibilities (In Execution Order)

- Tejas (Level 2 & 6 Defense Lead): Defends the 9-layer system blueprint, data trust perimeters, and the Next.js 16 SSE architecture; conducts the live UI demo.
- Dakshit (Level 3 Defense Lead): Defends tabular PaddleOCR extraction algorithms, Qwen-VL visual reasoning, and Qdrant HNSW vector retrieval.
- Nidhish (Level 1 & 3 Defense Lead): Defends refinery industrial context, SLM VRAM sizing, Adaptive Model Routing heuristics, and Constitutional Policy guardrails.
- Samar (Level 3 & 5 Defense Lead): Defends LangGraph StateGraph orchestration, PostgreSQL relational schemas, and Docker Compose orchestration.
- Shreya (Level 4 Defense Lead): Defends host iptables drop rules, /proc/net/tcp socket monitoring, and the forward-chained SHA-256 audit ledger.
- Safa (Level 4 & 5 Defense Lead): Defends Docker container sandboxing (`--network none`), Self-RAG verification mathematics (ISREL/ISSUP), and disaster recovery protocols.

Collaboration Flow

The team operates as a unified engineering defense unit. When a juror asks a cross-cutting question, the lead developer transitions smoothly to their peer, demonstrating deep team cohesion.

Prerequisites

- All 35 previous days completed.
- Complete system running cleanly on all 6 laptops.

Hands-On Requirement

Every developer stands before the simulated jury, draws their assigned architecture segment on a whiteboard/screen, answers 3 rigorous technical questions, and demonstrates live execution.

Knowledge Sharing

- Presenter: Entire Team
- Topic: "ABHEDYA AI Final Technical Defense: Demonstrating Sovereignty, Safety, and Agentic Intelligence for Confidential Industrial Operations."
- Duration: 60 minutes during Sunday online session.

Review

Final collective peer review: every team member verifies and signs off on the 18-point completion checklist.

Documentation

Commit final sign-offs to `developer_plan.md`, `docs/13-sih/evaluation-defense-faq.md`, and create final repository archive commit.

Expected Result

A production-grade, verifiably air-gapped sovereign AI workbench defended with complete technical mastery by all 6 developers.

Definition of Done

All 6 developers pass Level 1-6 jury cross-examination; all 18 checklist items signed off; repository tagged and ready for competition deployment.

21. Final Project Completion Checklist

- ✓ 1. Repository & Git Workflow: All six developers can independently clone, branch, commit, and push using standardized Conventional Commits and pre-commit hooks.
- ✓ 2. Environment Configuration: All six developers can configure local `.env` files, Python 3.11 virtual environments, and Node.js 20 runtimes without environment drift.
- ✓ 3. Local Stack Execution: All six developers can independently execute the complete platform locally (FastAPI backend + Next.js frontend + PostgreSQL + Qdrant + Ollama).
- ✓ 4. Architecture Comprehension: All six developers can draw and explain all 9 layers of the ABHEDYA AI architecture from memory.
- ✓ 5. Frontend Mastery: All six developers understand the Next.js 16 AppShell, Zustand state management, real-time SSE subscriptions, and responsive UI components.
- ✓ 6. Backend & Gateway Mastery: All six developers understand the FastAPI application lifecycle, async background task dispatching, and REST/SSE API endpoints.
- ✓ 7. Database & Migrations: All six developers understand the PostgreSQL relational schema, Alembic migration scripts, and repository CRUD patterns.
- ✓ 8. Agentic Orchestration: All six developers understand the LangGraph compiled StateGraph, node transformations, cyclic loops, and state serialization.
- ✓ 9. Local AI & Model Routing: All six developers understand local Ollama model management, the Adaptive Model Router heuristic, and Constitutional Policy guardrails.
- ✓ 10. Multimodal Ingestion & OCR: All six developers understand PaddleOCR tabular extraction from scanned NDT logs and Qwen2.5-VL visual defect analysis.
- ✓ 11. Sovereign Vector RAG: All six developers understand BGE-M3 local dense embeddings, Qdrant HNSW vector search, and tagged collection filtering.
- ✓ 12. Isolated Tool Sandboxing: All six developers understand Docker network-denied container execution (`--network none`), AST code scanning, and resource ceilings.
- ✓ 13. Self-RAG Verification: All six developers understand algorithmic ISREL and ISSUP critique calculations and automated revision loops.
- ✓ 14. Human-in-the-Loop Oversight: All six developers understand LangGraph checkpoint state freezing, operator approval interfaces, and cryptographic digital sign-offs.
- ✓ 15. Deliverable Synthesis: All six developers understand automated publication-quality `.docx` report generation with embedded tables, charts, and SHA-256 stamps.
- ✓ 16. Zero-Egress Security: All six developers can execute and defend iptables default-deny rules, `/proc/net/tcp` socket monitoring, and physical air-gap proof.
- ✓ 17. Immutable Audit Ledger: All six developers can explain and verify the forward-chained SHA-256 cryptographic audit ledger and demonstrate tampering detection.
- ✓ 18. Master Containerization & Handover: All six developers can execute single-command deployment via `docker compose up -d`, troubleshoot simulated faults, and successfully defend the system before a technical jury.

21. Final Project Completion Checklist

- ✅ 1. Repository & Git Workflow: All six developers can independently clone, branch, commit, and push using standardized Conventional Commits and pre-commit hooks.
- ✅ 2. Environment Configuration: All six developers can configure local .env files, Python 3.11 virtual environments, and Node.js 20 runtimes without environment drift.
- ✅ 3. Local Stack Execution: All six developers can independently execute the complete platform locally (FastAPI backend + Next.js frontend + PostgreSQL + Qdrant + Ollama).
- ✅ 4. Architecture Comprehension: All six developers can draw and explain all 9 layers of the ABHEDYA AI architecture from memory.
- ✅ 5. Frontend Mastery: All six developers understand the Next.js 16 AppShell, Zustand state management, real-time SSE subscriptions, and responsive UI components.
- ✅ 6. Backend & Gateway Mastery: All six developers understand the FastAPI application lifecycle, async background task dispatching, and REST/SSE API endpoints.
- ✅ 7. Database & Migrations: All six developers understand the PostgreSQL relational schema, Alembic migration scripts, and repository CRUD patterns.
- ✅ 8. Agentic Orchestration: All six developers understand the LangGraph compiled StateGraph, node transformations, cyclic loops, and state serialization.
- ✅ 9. Local AI & Model Routing: All six developers understand local Ollama model management, the Adaptive Model Router heuristic, and Constitutional Policy guardrails.
- ✅ 10. Multimodal Ingestion & OCR: All six developers understand PaddleOCR tabular extraction from scanned NDT logs and Qwen2.5-VL visual defect analysis.
- ✅ 11. Sovereign Vector RAG: All six developers understand BGE-M3 local dense embeddings, Qdrant HNSW vector search, and tagged collection filtering.
- ✅ 12. Isolated Tool Sandboxing: All six developers understand Docker network-denied container execution (--network none), AST code scanning, and resource ceilings.
- ✅ 13. Self-RAG Verification: All six developers understand algorithmic ISREL and ISSUP critique calculations and automated revision loops.
- ✅ 14. Human-in-the-Loop Oversight: All six developers understand LangGraph checkpoint state freezing, operator approval interfaces, and cryptographic digital sign-offs.
- ✅ 15. Deliverable Synthesis: All six developers understand automated publication-quality .docx report generation with embedded tables, charts, and SHA-256 stamps.
- ✅ 16. Zero-Egress Security: All six developers can execute and defend iptables default-deny rules, /proc/net/tcp socket monitoring, and physical air-gap proof.
- ✅ 17. Immutable Audit Ledger: All six developers can explain and verify the forward-chained SHA-256 cryptographic audit ledger and demonstrate tampering detection.
- ✅ 18. Master Containerization & Handover: All six developers can execute single-command deployment via docker compose up -d, troubleshoot simulated faults, and successfully defend the system before a technical jury.